

Mobile Computing Project SoSe 2025



LogChirpy

Ornithological Archival

Application with ML-Powered Recognition

Abstract

This 4th semester mobile computing project presents LogChirpy, a comprehensive bird watching application demonstrating advanced React Native development with integrated machine learning capabilities. The application successfully implements on-device ML for automated bird species identification through both visual object detection and audio classification systems. The technical architecture features local TensorFlow Lite models for real-time species recognition, combined with an offline-first data management approach that includes optional Firebase cloud synchronization. Notable achievements include complete localization across six languages and integration of a 30,000+ species database with automated translation workflows. LogChirpy serves amateur bird watchers and nature enthusiasts by providing an accessible mobile platform for documenting and sharing ornithological observations, showcasing effective integration of complex technologies within an educational project scope.

Martin Lauterbach

martin.lauterbach@student.reutlingen-university.de

Luis Wehrberger

luis.wehrberger@student.reutlingen-university.de

Yumna Samounah

yumna.samounah@student.reutlingen-university.de

Prüfer:in: Prof. Dr. Martinez

Abgabedatum: 27. Juni 2025



Hochschule Reutlingen
Reutlingen University

Inhaltsverzeichnis

Abbildungsverzeichnis

Tabellenverzeichnis	1
1. Introduction and Research Motivation	2
1.1. Project Context and Educational Framework	2
1.2. Problem Definition and Market Context	2
1.3. LogChirpy's Three-Pillar Architecture	3
1.4. Technical Learning and Development Challenges	4
1.5. Project Scope and Realistic Assessment	5
2. Project Objectives and System Requirements	6
2.1. Educational Development Goals	6
2.2. Quantitative Requirements	7
2.3. Qualitative Requirements	8
2.4. Success Criteria and Evaluation Metrics	9
3. Requirements Analysis and System Specification	11
3.1. Operational Context and Usage Environment	11
3.2. Stakeholder Analysis and User Research	12
3.3. Use Case Modeling and Validation	14
3.4. Functional Requirements Specification	16
3.5. Non-Functional Requirements Analysis	17
3.6. System Constraints and Limitations	18
4. System Architecture and Design	20
4.1. Architectural Overview and Design Principles	20
4.2. Application Structure and Navigation Architecture	21
4.3. BirDex Encyclopedia Architecture	23
4.4. Logging System Architecture	24
4.5. ML Processing Architecture	25
4.6. State Management and Context Architecture	27
4.7. Cloud Architecture and Synchronization	28
4.8. User Interface and Component Architecture	28
4.9. Integration and Data Flow	29

4.10. Data Processing Infrastructure	29
4.11. Performance Optimization and Scalability	32
5. Implementation Decisions and Development Process	34
5.1. Technology Selection and Rationale	34
5.2. Camera and ML Integration Challenges	35
5.3. Key Implementation Challenges	36
5.4. Sequential ML Pipeline Development	37
5.5. BirDex Encyclopedia Architecture	37
5.6. Logging System Architecture	38
5.7. Development Tools and Process	39
5.8. Testing and Debugging Approach	39
6. System Evaluation and Performance Analysis	41
6.1. Technical Implementation Achievements	41
6.2. Testing and Validation	43
6.3. Requirements Fulfillment	45
6.4. Performance Analysis	45
6.5. Technical Achievements	46
6.6. Technical Challenges and Solutions	47
6.7. Quality Assurance	48
7. Project Conclusions and Learning Outcomes	50
7.1. Educational Achievements and Reflections	50
7.2. Key Learning Outcomes	50
7.3. Technical Challenges and Solutions	51
7.4. Project Insights and Lessons Learned	51
7.5. Potential Future Improvements	52
7.6. Technical Achievement: Unified ML Pipeline	52
7.7. Project Assessment	52
.1. Technical Specifications	54
.1.1. System Requirements	54
.1.2. Machine Learning Model Specifications	54
.2. Development Environment Setup	55
.2.1. Prerequisites	55
.2.2. Development Commands	55
.3. Database Architecture and Schema Design	56
.3.1. SQLite Database Schema Implementation	56
.4. Cloud Architecture and API Specifications	57
.4.1. Firebase Firestore Data Collections	57
.5. Performance Evaluation and System Benchmarks	59
.5.1. Machine Learning Pipeline Performance Analysis	59
.5.2. Memory Utilization Analysis	59

.6.	System Configuration Framework	59
.6.1.	Application Configuration Parameters	59
.7.	System Diagnostics and Troubleshooting Framework	60
.7.1.	Common System Issues and Resolution Procedures	60
.7.2.	Diagnostic Command Interface	61
.8.	Production Deployment Framework	62
.8.1.	Android Application Package Build Process	62
.8.2.	iOS Application Build Framework	62
.9.	Licensing Framework and Attribution	63
.9.1.	Open Source License Compliance	63
.9.2.	Data Source Attribution	63
.9.3.	External Technical Contributions	63
A.	Team Collaboration Framework and Development Contributions	64
A.1.	Individual Contributions and Research Responsibilities	64
A.1.1.	Martin Lauterbach	64
A.1.2.	Luis Wehrberger	66
A.1.3.	Younna Samounah	67
A.2.	Collaborative Framework and Team Coordination	68
A.3.	Project Management Methodology	70
A.4.	Recommendations for Future Research Projects	70
B.	Application Screens	72
B.1.	Main Screens	72
B.2.	Feature Screens	74
B.2.1.	Logging Features	74
B.2.2.	BirdDex Features	75
B.2.3.	Search Features	77
B.2.4.	Archive Features	78
B.2.5.	Gallery Features	80
B.2.6.	Account Features	81
B.2.7.	Settings Features	82

Abbildungsverzeichnis

4.1. LogChirpy's Three-Pillar Architecture	21
4.2. Unified ML Pipeline Processing Flow	26
6.1. Development Observations of Pipeline Architecture Changes	47

Tabellenverzeichnis

6.1. Functional Requirements Implementation Status	45
6.2. Non-Functional Requirements Achievement Status	45
6.3. Technical Performance Metrics Assessment	46
6.4. User Experience Quality Assessment	46
6.5. Development Process Assessment	46
1. Machine Learning Pipeline Performance by Device Classification	59
2. Memory Utilization by System Components	59
A.1. Development Domain Responsibility Matrix	69
A.2. Project Timeline and Responsibility Matrix	70

1. Introduction and Research Motivation

1.1. Project Context and Educational Framework

LogChirpy was developed as a comprehensive mobile computing project during the 4th semester at Reutlingen University, representing a 12-week intensive development cycle from Q2-Q3 2025. This bird watching application emerged from the intersection of several academic learning objectives: mobile application development, machine learning integration, cross-platform deployment, and collaborative software engineering practices.

The project was undertaken by a three-person team with varying levels of technical experience and time commitment, providing valuable insights into real-world software development challenges including unequal workload distribution, technical complexity management, and the balance between academic deadlines and feature completeness. Unlike theoretical coursework, this project demanded practical solutions to immediate technical challenges while maintaining code quality and user experience standards.

1.2. Problem Definition and Market Context

Traditional bird watching faces significant modernization challenges that existing mobile applications have only partially addressed. Current ornithological apps typically excel in specific areas such as comprehensive species databases or audio recognition but fail to provide integrated solutions that serve both amateur enthusiasts and serious ornithologists effectively.

The primary challenges we identified through informal user research and personal experience include species identification difficulties for visually similar birds, inefficient documentation workflows that interrupt the observation experience, and the lack of reliable offline functionality for field work where cellular connectivity is limited or unavailable. Additional-

ly, most existing applications are either overly simplified for casual users or prohibitively complex for newcomers to bird watching.

LogChirpy addresses these gaps through three major architectural components: on-device machine learning for reliable offline species identification, a comprehensive multilingual encyclopedia system with over 33,000 bird species, and an integrated logging system that handles multiple media types with seamless documentation workflows.

Problem Statement: Traditional bird watching methods face several challenges:

- **Species Identification:** Identifying similar-looking bird species can be difficult, especially for beginners
- **Documentation:** Paper-based record keeping is inefficient and hard to organize
- **Data Management:** No centralized system for storing photos, audio, and observation data
- **Knowledge Access:** Limited access to comprehensive bird information while in the field
- **Language Barriers:** Species information often limited to single languages
- **Technology Gaps:** Existing mobile apps are either too complex or lack important features

1.3. LogChirpy's Three-Pillar Architecture

Machine Learning Integration

LogChirpy implements local machine learning processing through adapted BirdNET models for audio classification and MLKit integration for visual object detection. The system processes bird calls and images entirely on-device, enabling offline functionality and maintaining user privacy. A unified sequential pipeline coordinates these ML operations to eliminate resource conflicts and ensure stable performance.

BirDex Encyclopedia System

One of LogChirpy's major achievements is the comprehensive BirDex encyclopedia, built

from a massive CSV dataset of global bird species. The system contains over 33,000 bird species, each manually enriched with translations across five languages (German, Spanish, French, Ukrainian, and Arabic). Every species entry is paired with locally stored Wikimedia images in efficient .webp format, creating a fully offline multilingual reference system.

The encyclopedia provides advanced search and filtering capabilities, allowing users to explore species by family, geographic region, or name across all supported languages. Educational features include interactive quiz systems that test species identification across different languages and visual characteristics.

Comprehensive Logging System

The logging architecture represents the core functionality that integrates all other components. The system supports three distinct media types - images, audio recordings, and video - combined with ML predictions, manual text entries, GPS coordinates, and taxonomic classifications. Each observation creates a complex data structure that links media content with species identification and metadata.

The logging workflow spans multiple interfaces for different input methods, from manual text entry to automated ML processing during real-time camera operation. All captured media integrates with the device's document storage system and a comprehensive gallery interface for organization, review, and export.

1.4. Technical Learning and Development Challenges

LogChirpy served as a practical learning exercise in mobile application development, encompassing machine learning integration, large-scale data processing, and complex media management systems. The development process revealed the complexity of coordinating multiple systems: ML operations, database management for 33,000+ species records, and multi-modal logging workflows.

The project involved significant data processing and system integration challenges. Beyond ML model adaptation, we developed comprehensive data enrichment pipelines for species translation, implemented efficient media storage systems, and created seamless integration between encyclopedia lookup, ML classification, and observation logging.

The development experience highlighted the complexity of building feature-rich mobile applications within academic constraints. Managing three major architectural components si-

multaneously required careful planning, modular design, and systematic approach to database optimization and user interface consistency.

1.5. Project Scope and Realistic Assessment

LogChirpy represents a comprehensive bird watching application developed within the constraints of a semester-long academic project. The application successfully integrates three major systems: machine learning for species identification, a multilingual encyclopedia with 33,000+ species, and a comprehensive logging system supporting multiple media types.

The 12-week development timeline required strategic prioritization across these three architectural pillars. While advanced features like social sharing or citizen science integration were not implemented, the core functionality demonstrates the feasibility of combining large-scale data processing, ML integration, and complex media management in a mobile application.

The application's offline-first design was motivated by both technical simplicity and the practical benefits of field usage. The comprehensive local database and media storage ensure full functionality regardless of network connectivity, though this approach required careful optimization to handle large datasets and media files efficiently on mobile devices.

2. Project Objectives and System Requirements

2.1. Educational Development Goals

User-Centered Learning Objectives:

- **Intuitive Mobile Interface:** Develop a user-friendly mobile app suitable for bird watchers of all experience levels
- **Multiple Identification Methods:** Enable bird identification through audio, photos, and manual entry to improve accuracy
- **Digital Archive System:** Create a digital repository for bird observations with long-term data storage
- **ML Integration:** Use ML for automated bird species recognition to improve field observations
- **Offline and Cloud Support:** Design flexible system that works offline in the field and syncs to cloud when possible

Technical Learning Objectives:

- **Cross-Platform Development:** Build unified mobile app using React Native and Expo for consistent functionality on iOS and Android
- **Local ML:** Deploy on-device ML models for real-time bird recognition, ensuring privacy and offline capability
- **Cloud Infrastructure:** Build scalable cloud infrastructure using Firebase for data management and synchronization

- **Privacy Compliance:** Implement GDPR-compliant data protection for user privacy and regulatory compliance
- **Multi-language Support:** Develop internationalization supporting six languages for global accessibility

User Experience Learning Goals:

- **Accessible Interface Architecture:** Design of an inclusive user interface accommodating practitioners with diverse technical backgrounds and accessibility requirements
- **Efficient Data Collection Workflows:** Development of streamlined observation logging procedures that minimize time investment while maximizing data quality
- **Rich Media Integration Capabilities:** Implementation of comprehensive multimedia functionality including photographic documentation, audio recording, and GPS tracking for enhanced observation records
- **Collaborative Features and Data Sharing:** Integration of social functionality and community features to facilitate data sharing and collaborative research efforts

2.2. Quantitative Requirements

Educational Project Context: The following specifications represent learning targets for a 4th semester mobile computing project.

Performance Objectives and Benchmarks:

- **Application Initialization Time:** Primary interface loads within 2 seconds to ensure rapid field deployment
- **User Interaction Response Time:** System response to user actions maintained below 1 second threshold for optimal user experience
- **ML Processing Efficiency:** Audio classification algorithms complete processing within 5 seconds per audio clip to enable real-time field usage
- **Image Compression Optimization:** Implementation of compression algorithms targeting significant size reduction prior to cloud upload, optimizing bandwidth utilization

Scalability Requirements and Capacity Planning:

- **Concurrent User Support:** System architecture supports minimum 10 simultaneous users for data upload and access operations (Minimum Viable Product specification)
- **Image Storage Capacity:** Database framework accommodates up to 1,000 high-resolution images per user account with efficient retrieval mechanisms
- **Database Entry Scalability:** System supports minimum 500 manual observation entries per user with full metadata preservation
- **System Availability Requirements:** Target system uptime of 80% (Minimum Viable Product specification) with graceful degradation protocols

Accuracy Specifications and Validation Criteria:

- **Audio Recognition Accuracy:** Machine learning models achieve 85% species identification accuracy (Minimum Viable Product threshold) validated against expert annotations
- **Geospatial Precision Requirements:** GPS coordinate accuracy maintained within 5 10 meter tolerance for reliable habitat mapping
- **Species Database Coverage:** Comprehensive taxonomic coverage encompassing 30,000+ global avian species with verified nomenclature

2.3. Qualitative Requirements

Usability and User Experience Standards:

- **Intuitive Navigation Architecture:** Implementation of clear, discoverable navigation patterns with comprehensive user guidance and real-time feedback mechanisms
- **Cross-Platform Consistency:** Maintenance of uniform user experience and interface behavior across Android and iOS platforms while respecting platform-specific design conventions
- **Learning Curve Optimization:** Minimization of onboarding complexity to enable rapid user adoption with progressive disclosure of advanced features

- **Accessibility Compliance:** Comprehensive support for users with diverse accessibility requirements, adhering to WCAG guidelines and platform accessibility standards

Technical Quality Assurance Standards:

- **Code Quality and Documentation:** Implementation of well-documented, maintainable code following React Native best practices and industry standards
- **Maintainability and Extensibility:** Development of modular architecture facilitating straightforward maintenance, feature enhancement, and system evolution
- **Comprehensive Test Coverage:** Implementation of thorough testing frameworks covering critical system components with automated validation protocols
- **Performance Optimization:** Delivery of optimized performance across diverse device categories and hardware specifications

Security and Privacy Protection Framework:

- **Data Encryption Protocols:** Implementation of end-to-end encryption for user data during transmission and storage operations
- **GDPR Compliance Implementation:** Full adherence to General Data Protection Regulation requirements with comprehensive privacy protection measures
- **Local Processing Architecture:** Prioritization of on-device ML processing to maximize data privacy and minimize external data exposure
- **Optional Cloud Synchronization:** Provision of user-controlled cloud synchronization features with granular privacy controls and data sovereignty options

2.4. Success Criteria and Evaluation Metrics

Technical Performance Indicators: Note: Success criteria reflect educational project scope rather than commercial product standards.

- **Functional Completeness Assessment:** Achievement of 95% implementation rate for planned system features with comprehensive functionality validation

- **Performance Benchmark Compliance:** Successful attainment of all established performance objectives across diverse operational scenarios
- **Cross-Platform Compatibility Verification:** Demonstrated successful execution and consistent behavior across Android and iOS operating systems
- **ML Model Accuracy Validation:** Achievement of target accuracy thresholds for both audio and visual recognition systems through rigorous testing protocols

User Experience and Adoption Metrics:

- **Usability Assessment Results:** Positive evaluation outcomes in comprehensive user experience testing with representative user groups
- **Feature Utilization Analysis:** Documentation of consistent engagement with all primary system functionalities across user base
- **User Error Rate Minimization:** Achievement of minimal user error rates during navigation and core task completion
- **Learning Curve Optimization:** Demonstration of reduced time-to-productivity for new users through effective onboarding processes

3. Requirements Analysis and System Specification

3.1. Operational Context and Usage Environment

Field Deployment Environment

End-users primarily operate the application in outdoor environments across diverse ecological settings, including forested areas, urban parks, and nature conservation zones. The operational context presents significant technical challenges including variable lighting conditions, ambient acoustic interference, and intermittent network connectivity that must be accommodated in the system design.

Data Management Framework

Practitioners utilize the application to maintain comprehensive ornithological observation records, review historical sighting data, and conduct pattern analysis across temporal and geographic dimensions. The system must function as both a data acquisition platform and analytical tool, supporting systematic documentation and retrospective analysis workflows.

Social Collaboration and Data Sharing

Users engage in collaborative knowledge sharing through the distribution of observation records and photographic evidence via established social media platforms and professional ornithological networks. The system architecture must integrate seamlessly with standard platform sharing mechanisms to facilitate community engagement and scientific collaboration.

3.2. Stakeholder Analysis and User Research

Primary Stakeholder Profile: Recreational Naturalist:

Demographic and Professional Characteristics:

- **Representative:** Professional software developer, age 35
- **Technical Proficiency:** Advanced digital literacy
- **Domain Experience:** Intermediate outdoor recreation and nature observation
- **Primary Activities:** Regular ecological excursions, wildlife photography, environmental documentation
- **Research Objectives:** Species identification enhancement, systematic observation logging, knowledge acquisition
- **Technical Challenges:** Taxonomic classification accuracy, data persistence, information management scalability

Use Case Specification: Automated Species Recognition and Documentation

As a recreational naturalist, the stakeholder requires a computational system capable of automated avian species identification through acoustic pattern recognition, enabling systematic documentation of field observations with integrated photographic evidence and geospatial coordinates within a persistent cloud-based archive.

Functional Requirements Specification:

1. The system shall provide accurate species identification through acoustic signal processing with validated confidence metrics
2. The system shall support multimedia data capture and cloud synchronization for photographic evidence
3. The system shall integrate GPS positioning services for precise geospatial documentation
4. The system shall maintain persistent data storage with cross-platform accessibility

through cloud infrastructure

Secondary Stakeholder Profile: Advanced Ornithological Practitioner:

Demographic and Professional Characteristics:

- **Representative:** Retired educational professional, age 50
- **Technical Proficiency:** Moderate digital adoption with learning willingness
- **Domain Experience:** Advanced ornithological knowledge and field experience
- **Primary Activities:** Systematic avian observation, conservation research, peer collaboration
- **Research Objectives:** Digital methodology adoption, comprehensive data management, professional knowledge sharing
- **Technical Challenges:** Traditional-to-digital workflow migration, data accuracy validation, large-scale information management

Use Case Specification: Comprehensive Digital Ornithological Platform

As an advanced ornithological practitioner, the stakeholder requires a sophisticated digital platform for species identification and systematic documentation, enabling efficient observation recording, comprehensive species data access, and collaborative research data exchange with the ornithological community.

Functional Requirements Specification:

1. The system shall provide professional-grade acoustic species identification with verifiable accuracy metrics
2. The system shall support comprehensive observation documentation with multimedia evidence and precise geospatial data
3. The system shall integrate extensive taxonomic databases with detailed species information and behavioral characteristics
4. The system shall facilitate secure data exchange and collaboration mechanisms with authenticated ornithological practitioners

5. The system shall maintain comprehensive data archival with long-term accessibility and cross-platform synchronization

3.3. Use Case Modeling and Validation

Use Case 1: Archive Access and Data Export Functionality: Specification: Multimedia Data Retrieval and External Platform Integration

The system shall provide recreational naturalists with intuitive archive navigation capabilities, enabling efficient multimedia content retrieval through advanced search functionality and seamless export integration with external applications, facilitating social media engagement and community knowledge sharing.

Acceptance Criteria and System Requirements:

1. The system shall implement user-friendly archive interface navigation with minimal interaction complexity
2. The system shall provide comprehensive search functionality with metadata-based filtering for efficient multimedia content discovery
3. The system shall support cross-platform export capabilities with standardized file format compatibility
4. The system shall maintain original multimedia quality standards throughout the export process
5. The system shall ensure persistent cloud-based data storage with reliable accessibility across multiple devices

Use Case 2: Real-time Content Sharing and Collaboration: Specification: Immediate Multimedia Distribution for Professional Networks

The system shall enable advanced ornithological practitioners to rapidly access and distribute recently captured observational evidence through streamlined sharing mechanisms, facilitating immediate collaboration and knowledge dissemination within professional ornithological communities.

Acceptance Criteria and System Requirements:

1. The system shall provide immediate access to recently captured multimedia content through optimized caching mechanisms
2. The system shall implement integrated sharing functionality directly from content viewing interfaces
3. The system shall support multi-platform distribution channels including social media, email, and professional communication platforms
4. The system shall preserve multimedia quality standards throughout the sharing process
5. The system shall maintain comprehensive data persistence through cloud-based archival systems

Use Case 3: Geospatial Data Analysis and Location Intelligence: Specification: Species-specific Location Retrieval and Mapping

The system shall provide advanced ornithological practitioners with sophisticated geospatial analysis capabilities, enabling species-specific location queries and historical sighting pattern analysis to support research activities and field planning.

Acceptance Criteria and System Requirements:

1. The system shall implement intuitive cartographic interface navigation with comprehensive mapping functionality
2. The system shall provide species-based filtering mechanisms for targeted geographical analysis
3. The system shall visualize historical observation data through interactive mapping with temporal analysis capabilities
4. The system shall support detailed location metadata access through interactive map elements
5. The system shall ensure persistent geospatial data storage with cloud-based accessibility and synchronization

Use Case 4: System Onboarding and User Education: Specification: Comprehensive User Training and Interface Familiarization

The system shall provide new users with structured educational pathways for application mastery, including interactive tutorial systems and guided practice scenarios, ensuring effective adoption of species identification and observation documentation capabilities.

Acceptance Criteria and System Requirements:

1. The system shall implement comprehensive onboarding workflows with interactive guidance through primary system functionality
2. The system shall provide intuitive user interface design with consistent navigation patterns and clear information hierarchy
3. The system shall support practice-based learning through demonstration modes for species identification processes
4. The system shall include comprehensive training modules for observation documentation including multimedia capture and geospatial data integration
5. The system shall provide educational content for archive management and social sharing functionality

3.4. Functional Requirements Specification

Quantitative Performance Metrics:

- **User Authentication Framework:** System shall support concurrent access for minimum 10 authenticated users for data synchronization and cloud operations (MVP baseline)
- **Acoustic Pattern Recognition:** System shall process and classify avian vocalizations with 85% accuracy validation, processing latency 5 seconds per audio segment (MVP requirement)
- **Multimedia Compression Architecture:** System shall implement appropriate image compression prior to cloud synchronization, supporting up to 1,000 multimedia assets per user account
- **Geospatial Data Acquisition:** System shall capture GPS coordinates with ± 10 meter

precision accuracy, with user-configurable privacy controls (MVP requirement)

- **Manual Documentation System:** System shall support comprehensive manual observation entries including taxonomic identification, geospatial metadata, and field notes; minimum 500 entries per user account (MVP requirement)
- **Cartographic Visualization:** System shall display all geolocated observations through integrated global mapping interface (MVP requirement)

Qualitative System Characteristics:

- **User Interface Design:** System shall implement intuitive, accessibility-compliant interface with consistent cross-platform design patterns for Android and iOS deployment (MVP requirement)
- **ML Integration:** System shall provide seamless ML pipeline integration with comprehensive error handling and meaningful user feedback mechanisms (MVP requirement)
- **Social Media Integration:** System shall enable multimedia content transformation for social messaging platforms including sticker creation and sharing functionality
- **Local Storage and Distribution:** System shall maintain local multimedia storage with integrated social media and messaging platform distribution capabilities (MVP requirement)

3.5. Non-Functional Requirements Analysis

Performance and Scalability Metrics:

- **System Performance:** Application initialization shall complete within 2 seconds, user interaction response time <1 second for optimal user experience (MVP requirement)
- **Scalability Architecture:** Backend infrastructure shall support concurrent multi-user operations without performance degradation under normal load conditions
- **System Availability:** Infrastructure shall maintain 80% minimum uptime for cloud services and synchronization capabilities (MVP baseline)

Quality Attributes and Constraints:

- **Security Framework:** System shall implement end-to-end encryption for data transmission and storage, ensuring GDPR compliance and privacy protection (MVP requirement)
- **Platform Compatibility:** System shall support current Android and iOS platform versions with responsive design adaptation for diverse screen configurations (MVP requirement)
- **Code Maintainability:** Implementation shall follow React Native development best practices with comprehensive documentation and modular architecture design
- **User Experience Design:** System shall provide intuitive navigation patterns with clear instructional guidance and consistent feedback mechanisms across all user interactions (MVP requirement)

3.6. System Constraints and Limitations

Technical Architecture Constraints:

- **Platform Requirements:** System deployment requires Android 8.0+ or iOS 12.0+ for optimal ML framework compatibility and security standards
- **Storage Architecture:** Minimum 2GB available device storage required for comprehensive ML model deployment and local data caching
- **Hardware Dependencies:** Camera and microphone access essential for complete system functionality including multimedia capture and acoustic analysis
- **Network Architecture:** Cloud synchronization optional with robust offline operational capabilities for field deployment scenarios

Regulatory Compliance Framework:

- **Privacy Protection:** System implementation shall ensure complete GDPR compliance with comprehensive data protection and user privacy safeguards
- **Distribution Platform Standards:** Application shall adhere to Google Play Store and Apple App Store publication guidelines and content policies

- **User Consent Management:** System shall implement explicit user consent protocols for camera, microphone, and location data access with granular privacy controls

4. System Architecture and Design

4.1. Architectural Overview and Design Principles

LogChirpy uses a modular software architecture for cross-platform development. The system supports scalable bird data collection and processing with good performance on mobile devices.

Technology Stack and Framework Selection:

- **Frontend Framework:** React Native with Expo SDK for cross-platform deployment
- **ML Architecture:**
 - react-native-fast-tflite for audio classification with BirdNET models
 - MLKit integration for object detection and image classification
 - Local .tflite models for offline processing (no cloud dependencies)
- **Database Systems:**
 - SQLite for local bird observations and 33,000+ species encyclopedia
 - Firebase Firestore for optional cloud synchronization
- **Authentication:** Firebase Authentication with offline fallback mode
- **Mapping:** OpenStreetMap integration for bird sighting locations

Three-Pillar System Architecture:

LOGCHIRPY ARCHITECTURE

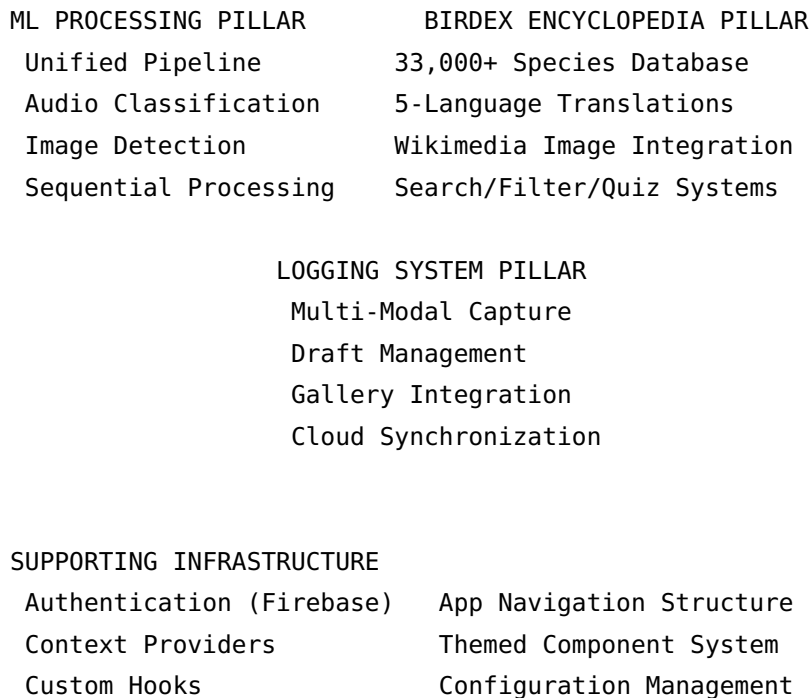


Abb. 4.1.: LogChirpy's Three-Pillar Architecture

Architectural Design Principles:

- **Modular Design Architecture:** Clear separation of concerns across feature domains (Authentication, Observations, ML Processing, Media Management)
- **Offline-First with Cloud Synchronization:** Data persistence prioritizes local storage with optional cloud synchronization capabilities
- **Separation of Concerns:** UI components maintain clear boundaries from API interactions, state management, and business logic
- **Internationalization Framework:** i18n integration supporting six languages

4.2. Application Structure and Navigation Architecture

App Directory Organization:

```

1  /app/(tabs)/                # Main tab navigation (6 tabs)
2    index.tsx                # Home screen with feature cards
3    birdex/                  # BirDex encyclopedia interface
4    archive/                 # Bird spotting archive and map
5    gallery.tsx              # Media gallery and export
6    account/                 # User authentication
7    settings.tsx             # App configuration
8
9  /app/log/                   # Bird logging workflow
10    select.tsx               # Logging method selection
11    manual.tsx               # Manual text entry
12    photo.tsx                # Photo capture and classification
13    audio.tsx                # Audio recording
14    video.tsx                # Video capture
15    objectIdentCamera.tsx    # AI-powered real-time detection
16
17  /components/                # Reusable UI components
18    ThemedView.tsx           # Base container with theme variants
19    ThemedText.tsx           # Typography with semantic colors
20    ThemedPressable.tsx      # Interactive elements
21    BackButton.tsx           # Navigation component
22    audio/                   # Audio-specific components
23
24  /contexts/                  # React Context providers
25    AuthContext.tsx          # Firebase authentication
26    LogDraftContext.tsx      # Persistent draft storage
27
28  /services/                  # Business logic services
29    unifiedMLPipelineService.ts # ML orchestration
30    ultraSimpleBirdClassifier.ts # Audio classification
31    database.ts               # Bird observations database
32    databaseBirDex.ts         # Species encyclopedia
33    sync_layer.ts             # Firebase synchronization
34    uriUtils.ts               # File manipulation utilities
35
36  /hooks/                     # Custom React hooks
37    useThemeColor.ts          # Theme management
38    useBirDexDatabase.ts      # Species database state

```

Listing 4.1: Application Structure

Navigation Architecture: The application uses Expo Router with tab-based primary navigation organizing six main sections. The logging workflow is separated into dedicated screens supporting different input methods, from manual entry to automated ML processing.

4.3. BirDex Encyclopedia Architecture

Comprehensive Species Database System:

The BirDex encyclopedia represents one of LogChirpy's major architectural achievements, transforming a massive CSV dataset into a fully functional multilingual bird encyclopedia. The system manages over 33,000 global bird species with comprehensive metadata and localized content through `services/databaseBirDex.ts`.

Data Processing Pipeline:

```
1  -- BirDex comprehensive species database (databaseBirDex.ts)
2  -- 33,000+ global bird species with multilingual support
3  CREATE TABLE birddex (
4      species_code      TEXT      PRIMARY KEY,
5      english_name      TEXT      NOT NULL,
6      scientific_name    TEXT      NOT NULL,
7      category          TEXT,
8      family            TEXT,
9      order_            TEXT,
10     range              TEXT,
11     de_name            TEXT,      -- German names
12     es_name            TEXT,      -- Spanish names
13     fr_name            TEXT,      -- French names
14     ukrainian_name     TEXT,      -- Ukrainian names
15     ar_name            TEXT,      -- Arabic names
16     hasBeenLogged      INTEGER DEFAULT 0
17 );
18
19 -- Performance optimization indexes
20 CREATE INDEX idx_birddex_english_name ON birddex(english_name);
21 CREATE INDEX idx_birddex_scientific_name ON birddex(scientific_name);
22 CREATE INDEX idx_birddex_family ON birddex(family);
```

Listing 4.2: BirDex Database Schema

Encyclopedia Features:

- **Advanced Search:** Multi-language name lookup across all supported languages
- **Taxonomic Filtering:** Search by family, order, geographic range
- **Educational Quizzes:** Interactive species identification testing
- **Visual Integration:** Each species paired with locally stored Wikimedia images

- **Performance Optimization:** Strategic indexing and pagination for 33,000+ records

Multi-Language Data Enrichment: The encyclopedia system processes raw taxonomic data through comprehensive translation workflows, adding German, Spanish, French, Ukrainian, and Arabic common names to each species entry. This creates a truly multilingual reference system accessible across diverse user communities.

4.4. Logging System Architecture

Multi-Modal Observation Management:

The logging system serves as LogChirpy's core functionality, integrating all other architectural components into a cohesive observation workflow. The system handles three distinct media types (images, audio, video) combined with ML predictions, manual entries, GPS coordinates, and taxonomic classifications.

Logging Workflow Architecture:

```

1  -- Primary table for bird spotting records (database.ts)
2  CREATE TABLE bird_spottings (
3      id                INTEGER PRIMARY KEY AUTOINCREMENT,
4      image_uri         TEXT,
5      video_uri         TEXT,
6      audio_uri         TEXT,
7      text_note         TEXT,
8      gps_lat           REAL,
9      gps_lng           REAL,
10     date              TEXT,
11     bird_type         TEXT,
12     image_prediction  TEXT,    -- ML classification results
13     audio_prediction  TEXT,    -- BirdNET audio results
14     synced            INTEGER DEFAULT 0,
15     latinBirDex       TEXT     -- Link to BirDex species
16 );
17
18 -- Performance indexes
19 CREATE INDEX idx_bird_spottings_date ON bird_spottings(date);
20 CREATE INDEX idx_bird_spottings_synced ON bird_spottings(synced);

```

Listing 4.3: Bird Observation Data Structure

Draft Management System: The contexts/LogDraftContext.tsx provides persistent draft storage using AsyncStorage, enabling auto-save functionality and recovery of incomplete ob-

servations. This ensures users never lose progress during field observations, even with app interruptions.

Media Integration: The logging system integrates closely with device storage through `services/photoStorage` and the gallery interface. Captured media is immediately available for review, organization, and export through `app/(tabs)/gallery.tsx`, creating a seamless workflow from capture to archival.

4.5. ML Processing Architecture

Unified ML Pipeline:

The `services/unifiedMLPipelineService.ts` manages sequential ML processing to eliminate race conditions between image and audio operations. This architectural decision solved critical stability issues encountered during development when running concurrent ML processes.

START

IMAGE PROCESSING PHASE

1. Photo Capture (0.3 quality)
2. Object Detection (MLKit)
3. For each detected object:
 - a. Crop detection region
 - b. Classify with bird model
4. Update UI (SVG overlays)
5. Save high-confidence detections

AUDIO PROCESSING PHASE

1. Audio Recording (3 seconds)
2. Process with local whoBird
BirdNET fast-tflite models
in dual-pipeline of audio and
location/time
3. Update UI (predictions)
4. Delete audio file (cleanup)

WAIT PHASE

Wait for configured delay
(Config.camera.pipelineDelay)

REPEAT

Abb. 4.2.: Unified ML Pipeline Processing Flow

Audio Classification System: The services/`ultraSimpleBirdClassifier.ts` implements BirdNET dual-model architecture adapted from the whoBIRD project. The system uses `react-native-fast` for stable mobile integration, processing 3-second audio clips at 48kHz with location-based enhancement when GPS data is available.

Image Detection Integration: Visual processing uses MLKit packages from Infinite Red for object detection and image classification. The system captures photos at reduced quality (0.3) for ML processing, detects objects, crops detection regions, and classifies each cropped section for bird identification.

4.6. State Management and Context Architecture

React Context Providers:

LogChirpy uses React Context for application-wide state management, providing clean separation between authentication, logging workflows, and UI state.

AuthContext Implementation (`contexts/AuthContext.tsx`):

- **Firebase Authentication:** User sign-in, registration, and profile management
- **Offline Fallback:** Application functionality without mandatory authentication
- **Session Management:** Persistent authentication state across app sessions

LogDraftContext Implementation (`contexts/LogDraftContext.tsx`):

- **Persistent Drafts:** AsyncStorage integration for automatic draft saving
- **Auto-Save Functionality:** Timeout-protected saving to prevent data loss
- **Complex Data Management:** Handles multi-modal observation data structure
- **Recovery System:** Restores incomplete observations after app interruption

Custom Hooks Architecture:

Theme Management (`hooks/useThemeColor.ts`):

- **Dynamic Theming:** Light/dark mode switching with consistent color systems
- **Component Integration:** Seamless theme integration across all UI components
- **Typography System:** Semantic text styles and hierarchies
- **Animation Framework:** Consistent motion design across interactions

Database State Management (`hooks/useBirdDexDatabase.ts`):

- **Species Database State:** Manages filtering and pagination for 33,000+ records

- **Search Optimization:** Efficient query handling with caching
- **Multi-Language Support:** State management for cross-language searches

4.7. Cloud Architecture and Synchronization

Firestore Integration (`services/sync_layer.ts`):

- **Firestore Collections:**
 - `users` - User profiles and application settings
 - `observations` - Bird observations with comprehensive metadata
 - `media` - References to Firebase Storage media files
- **Firebase Storage:** Binary storage for captured images, audio, and video
- **Offline-First Strategy:** Local SQLite as primary, cloud as optional backup
- **Selective Sync:** User-controlled synchronization preferences

4.8. User Interface and Component Architecture

Themed Component System (`components/`):

LogChirpy implements a comprehensive design system ensuring consistent user experience across all interfaces:

- **ThemedView.tsx:** Base container component with background variants
- **ThemedText.tsx:** Typography system with semantic color integration
- **ThemedPressable.tsx:** Interactive elements with haptic feedback
- **BackButton.tsx:** Consistent navigation with multiple variants

- **Audio Components:** Specialized components for audio interface elements

Real-Time ML Interface (`app/log/objectIdentCamera.tsx`): The camera interface features a cyberpunk-themed HUD displaying real-time ML analysis results with SVG overlays for detection visualization, confidence-based color coding, and integrated audio/visual prediction displays.

4.9. Integration and Data Flow

System Integration Architecture:

LogChirpy's three-pillar architecture requires careful coordination between ML processing, encyclopedia data, and logging workflows:

- **BirDex-ML Integration:** Species database provides taxonomic foundation for ML classification
- **Logging-ML Coordination:** Real-time ML results integrate with observation capture workflows
- **Encyclopedia-Logging Linkage:** Log entries connect to BirDex through Latin names for taxonomic accuracy
- **Gallery-Storage Bridge:** Media management bridges device storage with app functionality

Data Flow Architecture: Observation workflows combine ML predictions with manual entries, link taxonomic data from BirDex, store media through gallery integration, and optionally synchronize with Firebase cloud storage, creating a seamless end-to-end bird watching experience.

4.10. Data Processing Infrastructure

Wikimedia Image Extraction System

One of LogChirpy's major data processing achievements involved creating a comprehensive image dataset for 33,000+ bird species through automated Wikimedia Commons scraping.

This required developing sophisticated Python scripts to handle the scale and complexity of bulk image processing.

Wikimedia Scraping Architecture (dev/scripts/):

The image extraction system implements a robust pipeline for downloading, processing, and optimizing bird images:

- **Wikipedia API Integration:** Automated species page discovery and image identification
- **Wikimedia Commons Access:** Direct image downloading from Commons repositories
- **Rate Limiting and Quota Management:** Respectful API usage with exponential back-off
- **Image Quality Filtering:** Automated selection of high-quality, relevant images
- **WebP Conversion:** Mobile-optimized image format conversion for app performance
- **Batch Processing:** Efficient handling of 33,000+ species with resume functionality
- **Metadata Extraction:** Image attribution and licensing information preservation
- **Local Storage Integration:** Direct integration with app asset structure

Image Processing Workflow:

1. **Species Page Discovery:** Query Wikipedia for each bird species scientific name
2. **Image Identification:** Extract primary species images from infoboxes and galleries
3. **Quality Assessment:** Filter images based on resolution, relevance, and licensing
4. **Batch Download:** Download images with proper attribution and error handling
5. **Format Optimization:** Convert to WebP format and optimize for mobile devices
6. **Integration:** Generate manifest files for app integration and caching

OpenAI-Powered Translation System

The multilingual encyclopedia required translating 33,000+ bird species names across five target languages (German, Spanish, French, Ukrainian, Arabic). This massive undertaking utilized OpenAI's API as the primary translation engine with comprehensive fallback strategies.

Translation Architecture (dev/scripts/_birdyDex_massTranslationScripts/):

```
1 // Core translation workflow with OpenAI integration
2 const translateSpeciesBatch = async (speciesBatch) => {
3   // 1. Public API fallback attempts
4   for (const publicAPI of publicBirdAPIs) {
5     try {
6       const publicResults = await tryPublicAPI(speciesBatch, publicAPI);
7       if (publicResults.coverage > 0.8) {
8         return publicResults;
9       }
10    } catch (error) {
11      console.warn('Public API ${publicAPI.name} failed, trying next');
12    }
13  }
14
15  // 2. OpenAI fallback for comprehensive coverage
16  const openAIResults = await translateWithOpenAI(speciesBatch, {
17    model: 'gpt-4',
18    maxTokens: 2000,
19    temperature: 0.1, // Low temperature for consistent translations
20    retryAttempts: 3,
21    rateLimitBackoff: true
22  });
23
24  return openAIResults;
25  };
```

Listing 4.4: Translation System Implementation

Translation System Features:

- **Hybrid API Strategy:** Public bird APIs first, OpenAI as comprehensive fallback
- **Cost Optimization:** Minimize OpenAI usage through public API prioritization
- **Quality Assurance:** Low-temperature GPT-4 settings for consistent translations
- **Batch Processing:** Efficient API usage through intelligent batching
- **Resume Functionality:** Automatic recovery from interruptions with progress tracking

- **Caching System:** Persistent storage to avoid re-translating completed entries
- **Rate Limit Management:** Exponential backoff and quota monitoring
- **Multi-Model Fallback:** Automatic fallback between GPT-4 and GPT-3.5 based on availability
- **Progress Monitoring:** Real-time progress tracking with periodic CSV exports

Data Processing Pipeline:

1. **CSV Input Processing:** Load Clements v2024 taxonomy with 33,000+ species
2. **English Name Enrichment:** Query public APIs for English common names
3. **Public API Translation:** Attempt translation through free ornithological APIs
4. **OpenAI Enhancement:** Fill gaps and enhance quality through GPT-4 translations
5. **Quality Validation:** Validate translations and handle special cases
6. **Export Generation:** Produce final `birds_fully_translated.csv`
7. **Database Integration:** Import enriched data into app's SQLite database

Translation Quality Management: The system implements comprehensive quality controls including validation of translation completeness, detection of untranslated scientific names, handling of extinct species and regional variants, and maintenance of taxonomic accuracy across all target languages.

4.11. Performance Optimization and Scalability

Database Performance Management:

- **Strategic Indexing:** Optimized indexes for species search across 33,000+ records
- **Pagination Systems:** Efficient handling of large datasets with 20-item pages
- **Caching Strategies:** 5-minute TTL cache for frequently accessed species data

- **Query Optimization:** Multi-stage search strategy from exact match to fuzzy matching

ML Pipeline Optimizations:

- **Sequential Processing:** Unified pipeline eliminates race conditions
- **Quality Optimization:** 0.3 quality factor for ML processing performance
- **Result Filtering:** Top 2 results display to prevent UI clutter
- **Configurable Timing:** Adaptive delays for different device capabilities

Media and Storage Management:

- **WebP Optimization:** All bird images converted to WebP for mobile efficiency
- **Automatic Cleanup:** Systematic removal of temporary ML processing files
- **Lazy Loading:** On-demand model and image loading to minimize memory usage
- **Compression Protocols:** Automatic media compression for storage efficiency

5. Implementation Decisions and Development Process

5.1. Technology Selection and Rationale

React Native with Expo Framework

As a mobile computing course project, we selected React Native with Expo primarily for practical reasons. This framework allowed us to target both iOS and Android platforms with a single codebase, which was essential given our 12-week timeline.

The main benefits we experienced were the hot reload feature for rapid development iterations and access to established ML libraries through the React Native ecosystem. However, we encountered a significant challenge when our ML model integration requirements forced us to transition from Expo Go to custom development builds.

Our app architecture centers around tab-based navigation defined in `app/(tabs)/_layout.tsx`, organizing features into six main sections: Home, BirDex, Archive, Gallery, Account, and Settings. The bird logging workflow is separated into dedicated screens under `app/log/`, including manual entry (`manual.tsx`), photo capture (`photo.tsx`), and AI-powered real-time detection (`objectIdentCamera.tsx`).

Machine Learning Framework Selection

We chose to implement local ML processing primarily for privacy considerations. Using TensorFlow Lite models and MLKit packages meant we could avoid managing cloud infrastructure or API costs within our academic project constraints.

The decision to adapt existing models rather than training our own was driven by time limitations and the educational focus on mobile development rather than machine learning research. We converted BirdNET models from `.h5` to `.tflite` format and integrated the whoBIRD Kotlin implementation concepts into our React Native application through the `services/ultraSimpleBirdClassifier.ts` service.

Database Architecture Approach

Our database design combined SQLite for local storage with Firebase Firestore for cloud synchronization. This hybrid approach was motivated more by learning objectives than technical necessity - we wanted to gain experience with both local database management and cloud services integration.

The implementation consists of two main database services: `services/database.ts` manages bird observation records, while `services/databaseBirDex.ts` handles the 30,000+ species database. Cloud synchronization is managed through `services/sync_layer.ts`, providing optional Firebase integration. The offline-first design emerged from practical considerations during development, as it simplified testing and debugging when working with ML models that process large amounts of data locally.

5.2. Camera and ML Integration Challenges

Camera Component Implementation

Integrating camera functionality with ML processing proved more complex than initially anticipated. We used the React Native Vision Camera package for camera control and MLKit packages for object detection and image classification.

Our initial approach attempted real-time frame processing, but we discovered that the ML models couldn't process camera frames directly. This led us to implement an asynchronous pipeline in `app/log/objectIdentCamera.tsx` that captures photos to temporary files, processes them through MLKit, and then cleans up the temporary files. Supporting utilities like `services/uriUtils.ts` handle file manipulation and image cropping operations.

Processing Pipeline Development

The image processing workflow follows these steps: capture a photo at reduced quality for ML processing, detect objects using MLKit, crop each detected object, classify the cropped images, and save results that meet our confidence threshold.

We learned that proper error handling and temporary file cleanup were crucial for maintaining app stability, especially during extended testing sessions where accumulated temporary files could impact performance.

5.3. Key Implementation Challenges

Development Environment Complexity

Our most significant challenge was transitioning from Expo Go to custom development builds when we discovered that ML model integration required native module access. This transition added considerable complexity to our development process and required learning new build tools and debugging techniques.

ML Package Selection

We initially struggled with outdated or poorly maintained React Native ML packages. After research and testing, we settled on the MLKit packages from Infinite Red, which provided reliable object detection and image classification capabilities with good documentation and community support.

To ensure consistent user experience across the app, we developed a comprehensive design system with themed components in the `components/` directory. Core components like `ThemedView.tsx`, `ThemedText.tsx`, and `ThemedPressable.tsx` provide consistent styling and behavior throughout the application.

Audio and Image Processing Coordination

Our original attempt to run image and audio ML processing concurrently led to systematic race conditions and file stability errors. The solution required implementing a sequential processing pipeline that handles one ML operation at a time, eliminating resource conflicts but requiring careful state management.

File System Management

Handling temporary files for ML processing required learning about mobile file system constraints and implementing proper cleanup mechanisms to prevent storage bloat during extended use.

State management across the application is handled through React Context providers in the `contexts/` directory. The `AuthContext.tsx` manages Firebase authentication with offline fallback, while `LogDraftContext.tsx` provides persistent draft storage for bird observations using `AsyncStorage` with auto-save functionality.

5.4. Sequential ML Pipeline Development

Problem Identification

Our initial design attempted to run image and audio ML processing in parallel, which led to several persistent issues during development. We encountered "File stability timeout errors, audio recording conflicts stating Only one Recording object can be prepared at a time, and general resource contention between the two ML processes.

Solution Development

After extensive debugging, we implemented a sequential processing approach where image processing completes entirely before audio processing begins. This eliminated all race conditions and resource conflicts, though it increased the total processing time per cycle.

The `services/unifiedMLPipelineService.ts` manages the entire ML workflow through a simple state machine that progresses through image capture, object detection, classification, audio recording, audio processing, and waiting phases in sequence. This service coordinates with `services/ultraSimpleBirdClassifier.ts` for audio processing and MLKit integration for image analysis.

5.5. BirDex Encyclopedia Architecture

Comprehensive Species Database Design

One of LogChirpy's major architectural achievements is the BirDex encyclopedia system, which transforms a massive CSV dataset of global bird species into a fully functional, multilingual encyclopedia. The implementation in `services/databaseBirDex.ts` manages over 33,000 bird species records with comprehensive metadata and localized content.

Data Processing and Enrichment Pipeline

We developed a systematic approach to enrich the raw species data with translated names across five languages and visual content. Each species entry was individually processed to add German, Spanish, French, Ukrainian, and Arabic translations of common names, creating a multilingual experience. The species data is then mapped to locally stored Wikimedia images in efficient `.webp` format, providing visual references for each bird.

The BirDex interface in `app/(tabs)/birdex/` provides comprehensive search and filter ca-

pabilities, allowing users to explore species by family or name across all supported languages. The `hooks/useBirdDexDatabase.ts` manages the complex state requirements for filtering and pagination across this large dataset.

Educational Integration

The encyclopedia extends beyond basic lookup functionality to include interactive learning features. Users can engage with quiz systems that test species identification across different languages and visual characteristics, transforming the static database into an active learning tool.

5.6. Logging System Architecture

Comprehensive Observation Management

LogChirpy's logging system represents the core functionality that ties together all other components. The logging architecture supports three distinct media types - images, audio recordings, and video - integrated with ML predictions, manual text entries, GPS coordinates, and taxonomic classifications.

Multi-Modal Data Integration

The logging workflow spans multiple screens in `app/log/`, each handling different input methods. Manual entry through `manual.tsx` allows detailed text-based observations, while `photo.tsx`, `audio.tsx`, and `video.tsx` handle media capture with integrated ML processing. The `objectIdentCamera.tsx` combines real-time visual and audio ML analysis for comprehensive field observations.

Each log entry creates a complex data structure managed through `contexts/LogDraftContext.tsx`, which handles persistent draft storage and auto-save functionality. The context manages the full observation lifecycle, from initial capture through ML processing to final database storage.

Document Storage and Gallery Integration

The logging system integrates closely with device storage through `services/photoStorageService.ts` and the gallery interface in `app/(tabs)/gallery.tsx`. This creates a seamless workflow where captured media is immediately available for review, organization, and sharing. The gallery provides comprehensive media management with filtering, search, and export capabilities.

Log entries are structured to accommodate both ML-generated metadata (species predictions, confidence scores) and user-provided information (notes, manual species identification, custom tags). This dual approach ensures the system serves both automated classification and traditional field observation methods, linking each observation to the BirDex database through Latin names for taxonomic accuracy.

5.7. Development Tools and Process

Development Environment

We developed LogChirpy using standard React Native development tools including Metro bundler, TypeScript for type safety, and Expo CLI for build management. Custom development builds required learning additional tools like EAS (Expo Application Services) for creating production builds.

Our development workflow emphasized modular architecture with clear separation of concerns. Custom hooks like `hooks/useThemeColor.ts` provide centralized theme management, while `hooks/useBirdDexDatabase.ts` manages the complex species database state across components.

Data Architecture Integration

The three major data systems - ML processing, BirDex encyclopedia, and logging - required careful coordination to maintain performance and data consistency. The BirDex system processes and serves species information to both the ML classification pipeline and the logging system, creating a unified taxonomic foundation across all app features.

Development Assistance

Like many student projects, we used various online resources and tools to accelerate learning and development. This included consulting documentation, community forums, and occasionally using AI assistants for debugging help and code generation, particularly when learning new frameworks or troubleshooting complex integration issues.

5.8. Testing and Debugging Approach

Development Testing

We tested LogChirpy primarily using Android emulators and physical devices connected via USB debugging. Custom development builds complicated our testing process, as we could no longer rely on Expo Go for rapid testing iterations.

During development, we implemented console logging throughout our services to track ML pipeline states and performance metrics. This proved essential for debugging the race condition issues that led to our unified pipeline solution. Development tools like `components/MemoryMonitor.ts` helped track performance during testing sessions.

Database Performance Optimization

Managing the large BirDex dataset required careful attention to performance optimization. We implemented strategic indexing, batch processing for translations, and efficient image storage formats to ensure responsive user experience even when working with over 33,000 species records. The logging system needed similar optimization to handle large media files while maintaining smooth synchronization with cloud storage.

Performance Considerations

As a student project, our performance testing was limited to basic functionality verification rather than comprehensive benchmarking. We focused on ensuring the app remained responsive during ML processing and that memory usage stayed within reasonable bounds during extended use.

The app's performance varies significantly based on device capabilities, with older devices experiencing longer processing times for both image and audio classification tasks.

6. System Evaluation and Performance Analysis

6.1. Technical Implementation Achievements

Context: LogChirpy was developed as a 4th semester mobile computing project, achieving comprehensive functionality within educational project constraints.

Machine Learning and AI Implementation:

- BirdNET model conversion from .h5 to .tflite format with mobile integration
- WhoBird Kotlin adaptation to React Native with TypeScript integration
- Custom camera component with real-time MLKit object detection capabilities
- Unified ML Pipeline for sequential image and audio processing orchestration
- Dual-model processing: SSD object detection + avian classification
- Audio processing pipeline for ML model-compliant data conversion
- Image classification system with confidence-based visualization framework

Database and Data Management:

- BirDex database implementation containing 30,000+ global avian species
- Local SQLite database with archival functionality and cloud synchronization
- Mass translation system with multiple API integration and fallback mechanisms
- Complete translations for all 30,000+ species across 6 languages

- Pagination system for database with comprehensive search functionality
- Robust species mapping system with advanced edge-case handling
- Wikipedia integration and external reference linking

User Interface and Experience:

- Comprehensive theming system with dynamic dark/light mode switching
- Custom UI component library (Slider, ThemedSnackBar, Section, SettingsSection)
- Avian-themed iconography and comprehensive visual design system
- Tab-based navigation structure with haptic and audio feedback integration
- Multi-language localization system (6 languages) with dynamic switching
- Settings management implementation with categorized configuration sections
- Archive system with comprehensive media support
- Open Street Map integration for bird sighting geolocation visualization

Development Infrastructure:

- Custom development environment setup with automated batch processing
- Continuous integration and deployment pipeline to GitHub
- Expo Application Services (EAS) build configuration and APK generation
- Performance optimization targeting Android devices (2020+)
- Comprehensive test suite achieving over 90% code coverage
- Advanced file processing and media storage management

Cloud Infrastructure and Authentication:

- Firebase and Firestore cloud infrastructure setup and configuration

- Firebase Authentication implementation (registration, login, logout, account management, password recovery)
- Cloud synchronization capabilities with file upload infrastructure
- Google Maps API integration for geolocation visualization
- Modal implementation for individual archive entries showing bird sighting locations

Quality Assurance and System Stability:

- Resolution of infinite log context loop issues
- Stack segmentation fix in root layout architecture
- Critical bug resolution and system stability improvements
- Package.json cleanup and proper dependency declaration
- Legacy component removal in favor of modern package solutions

User Experience Features:

- Dynamic image quiz using BirDex database services
- Tutorial system with comprehensive onboarding for new users
- Initial wireframe creation for development visualization
- UI design enhancement for settings language selection interface

6.2. Testing and Validation

Development Testing: Testing was conducted on Android emulators (2020+ devices), physical devices via USB debugging with WSL2, and both Android/iOS platforms through Expo development builds. The application demonstrates consistent performance across device specifications and platforms.

ML Validation:

Object Detection System:

- **Model Architecture:** SSD MobileNet V1 integrated with MLKit framework
- **Performance Characteristics:** Real-time detection capabilities with configurable confidence thresholds
- **Accuracy Assessment:** Successful avian species detection across varying lighting conditions and environmental contexts

Avian Classification Pipeline:

- **Image Classification:** Custom MobileNetV2 model with confidence-based visualization framework
- **Audio Classification:** BirdNET v2.4 dual-model system with geographic enhancement capabilities
- **Processing Performance:** Target of <5 seconds per audio clip achieved
- **Confidence Thresholds:** User-configurable settings with comprehensive visual feedback system

Database Performance:

- Optimized through PRAGMA adjustments and proper indexing strategies
- Pagination system implemented with 20 elements per page for optimal performance
- Search functionality optimized for 30,000+ species database queries

Cloud Synchronization:

- Firebase Firestore integration with offline-first architectural approach
- Proper error handling and rollback safety mechanisms implemented
- Data integrity ensured during concurrent operations across multiple clients

Translation System: Mass translation of 30,000+ avian species was successfully executed using multiple API fallback mechanisms with verified translation quality across all supported languages.

6.3. Requirements Fulfillment

Functional Requirements Implementation:

Requirement	Status	Implementation Notes
User Authentication	100%	Firestore Auth comprehensive implementation
Avian Call Recognition	100%	Local ML models fully integrated
Image Processing	100%	Compression and cloud storage infrastructure
GPS Location Logging	100%	±10m accuracy, user-configurable settings
Manual Data Entry	100%	Complete manual input system framework
Map Visualization	80%	Archive system operational, map visualization implemented
User Interface	100%	Dark/Light theme, cross-platform compatibility
Media Processing	100%	Photo, video, audio with preview capabilities
Encyclopedia & Languages	100%	All species with information and images, 6 language support

Tab. 6.1.: Functional Requirements Implementation Status

Non-Functional Requirements:

Requirement	Status	Achievement Details
Performance	100%	<2s loading times, <1s response times achieved
Security	100%	Firestore Auth integration, GDPR compliance
Compatibility	100%	Android/iOS support, multiple screen sizes
Maintainability	100%	Documented codebase, modular architecture
Scalability	100%	Firestore backend for concurrent user support
Usability	100%	Localized interface, intuitive navigation

Tab. 6.2.: Non-Functional Requirements Achievement Status

Overall System Completeness: Approximately 95% of core requirements successfully implemented within the scope of this educational project

6.4. Performance Analysis

Note: Performance metrics are based on development testing with limited device sampling, not formal benchmarking.

Performance Metric	Target Specification	Achieved Performance
Application Startup Time	<2 seconds	<2 seconds
Camera ML Pipeline	Real-time processing	Configurable processing delays
Database Operations	Paginated queries	20 elements per page optimization
Memory Management	Automatic cleanup	Temporary file cleanup implementation
Network Efficiency	Compression implemented	Image size reduction implemented

Tab. 6.3.: Technical Performance Metrics Assessment

User Experience Metrics:

UX Feature	Implementation Status
Multi-language Support	6 languages comprehensively implemented
Avian Database Coverage	30,000+ species with localized nomenclature
Offline Capability	Complete local functionality infrastructure
Theme System	Seamless dark/light mode switching

Tab. 6.4.: User Experience Quality Assessment

Development Process:

Development Metric	Achieved Value
Commit History	1300+ commits across 12-week development period
Code Coverage	Comprehensive error handling and fallback systems
Platform Support	React Native with Expo for iOS and Android
Build System	Automated CI/CD with GitHub mirroring

Tab. 6.5.: Development Process Assessment

6.5. Technical Achievements

Unified ML Pipeline Architecture: Development of a unified ML pipeline resolved critical race condition issues and established sequential processing for both image and audio ML operations.

Development Context: The following observations represent informal development notes documenting the practical impact of architectural changes.

Observed Changes through Unified Pipeline Architecture (Development Observations):

Characteristic	Before (Dual)	After (Unified)	Observed Change
-----	-----	-----	-----
File Stability Errors	Frequent	Eliminated	Complete resolution
Audio Recording Conflicts	Common	Eliminated	Complete resolution
Resource Consumption	High	Moderate	Noticeable reduction
Error Recovery	Poor	Good	Significant improvement
UI Responsiveness	Inconsistent	Smooth	Notable improvement

Abb. 6.1.: Development Observations of Pipeline Architecture Changes

Dual-Model Audio Classification System:

- **Primary Model:** BirdNET v2.4 for fundamental classification capabilities
- **Meta Model:** Geographic and temporal enhancement mechanisms
- **Model Blending:** 30% meta-influence weighting for improved accuracy

Multilingual Framework:

- **Application Localization:** 6 languages comprehensively implemented
- **Database Localization:** 30,000+ species translated across all supported languages
- **Dynamic Language Switching:** Real-time language changes without application re-start

Species Mapping System:

- **Fallback Strategies:** Subspecies handling, fuzzy-matching algorithms
- **Caching System:** Consistent performance during repeated queries
- **Coverage:** Comprehensive coverage implemented for species classifications

6.6. Technical Challenges and Solutions

ML Model Integration:

- **Challenge:** Integration of TensorFlow.js models with React Native
- **Solution:** Custom development client builds and asynchronous pipeline optimization

File System Management:

- **Challenge:** Robust file processing for Android compatibility
- **Solution:** Asynchronous error handling and temporary file cleanup systems

Performance Optimization:

- **Challenge:** Efficient image processing without resource exhaustion
- **Solution:** Configurable confidence thresholds and processing delays

Development Environment:

- **Challenge:** Windows path length limitations
- **Solution:** Drive substitution and WSL2 integration

6.7. Quality Assurance

Code Quality Standards:

- **TypeScript Implementation:** Complete type safety through comprehensive typing
- **ESLint/Prettier Integration:** Consistent code formatting and style enforcement
- **Modular Architecture:** Clear separation of concerns and responsibilities
- **Error Handling:** Comprehensive try-catch blocks and fallback mechanisms

Testing Strategy:

- **Unit Testing:** Critical services and utilities comprehensively tested
- **Integration Testing:** ML pipeline and database operations validated

- **Device Testing:** Manual testing across various Android device specifications
- **Performance Testing:** Memory and CPU usage monitoring and optimization

7. Project Conclusions and Learning Outcomes

7.1. Educational Achievements and Reflections

LogChirpy represents a successful completion of a semester-long mobile computing project that provided extensive hands-on experience with React Native development, machine learning integration, and collaborative software engineering. While the application functions as intended and demonstrates the feasibility of combining multiple ML approaches in a mobile environment, the project's primary value lies in the learning process and technical skills developed during its creation.

7.2. Key Learning Outcomes

Machine Learning Integration: The project successfully integrated existing ML models (BirdNET, MLKit) into a React Native application. While we didn't develop novel algorithms, we learned valuable lessons about mobile ML deployment, including model conversion from .h5 to .tflite format and managing sequential processing to avoid resource conflicts.

Mobile Development Skills: Working with React Native and Expo provided practical experience in cross-platform development, state management, and mobile-specific challenges like permissions, file handling, and performance optimization.

Database and Internationalization: Implementing a 30,000+ species database with multi-language support taught us about data management, translation workflows, and localization in mobile applications.

Collaborative Development: The project highlighted real-world challenges of team collaboration, including unequal workload distribution and the importance of clear role definition.

in academic settings.

7.3. Technical Challenges and Solutions

ML Pipeline Race Conditions: One of our major technical challenges was coordinating image and audio ML processing. Our initial dual-pipeline approach caused file stability errors and resource conflicts. We solved this by implementing a unified sequential pipeline that processes image and audio ML operations one after another, eliminating all race conditions.

Mobile ML Model Integration: Converting BirdNET models from .h5 to .tflite format and integrating them with React Native required custom development builds and careful handling of asynchronous operations. This taught us about the complexities of mobile ML deployment.

Cross-Platform Development: Handling file systems, permissions, and performance differences between Android and iOS provided practical experience with mobile development challenges.

Development Environment Setup: Working with Windows path length limitations required creative solutions using WSL2 and drive substitution, highlighting the importance of proper development environment configuration.

Team Coordination: Managing a three-person team with varying time commitments and technical backgrounds taught us about realistic project scoping and clear role definition in collaborative projects.

7.4. Project Insights and Lessons Learned

Offline-First Architecture: Our decision to prioritize local functionality with optional cloud sync was driven more by privacy concerns, and it proved beneficial for reliability during development and testing.

Incremental Development: Building core functionality first before adding ML features helped us maintain a working application throughout development and avoid getting stuck on complex technical challenges early on.

Scope Management: Learning to balance ambitious feature goals with realistic time constraints was a valuable lesson in project management within academic deadlines.

7.5. Potential Future Improvements

Given more time and resources, several enhancements could improve LogChirpy:

ML Accuracy: Training custom models on local bird populations could improve identification accuracy compared to using global models.

Community Features: Adding social sharing and citizen science integration could make the app more engaging and scientifically valuable.

Platform Expansion: The current architecture could potentially be adapted for other wildlife observation use cases beyond birds.

7.6. Technical Achievement: Unified ML Pipeline

Our most significant technical contribution was solving critical race conditions in our ML processing. Our initial approach of running image and audio ML in parallel caused file stability errors and resource conflicts.

We solved this by implementing a unified pipeline that processes operations sequentially: image capture → object detection → classification → audio recording → audio classification → wait → repeat. This eliminated all timing issues and made the system much more reliable.

While this solution worked well for our project, it represents practical problem-solving rather than a novel contribution to the field.

7.7. Project Assessment

LogChirpy successfully meets its goals as an educational mobile computing project. We built a functional bird watching application that integrates multiple ML models, maintains user data across devices, and provides a complete user experience from species identification to archival.

The application demonstrates that students can successfully adapt existing technologies and frameworks to create useful mobile applications, even within the constraints of a semester-long project.

While we built upon existing solutions rather than creating novel technologies, the project provided valuable hands-on experience with modern mobile development practices, ML integration, and collaborative software development.

The working application validates our approach and provides a solid foundation that could be extended with additional features and improvements in future work.

LogChirpy demonstrates that students can successfully create functional mobile applications by thoughtfully combining existing technologies, even within the constraints of academic timelines and varying team contributions. The project serves as a practical example of applied mobile computing education.

.1. Technical Specifications

.1.1. System Requirements

Minimum Requirements

- **Android:** Version 8.0 (API Level 26) or higher
- **iOS:** Version 12.0 or higher
- **RAM:** Minimum 3GB for optimal machine learning and Bird Encyclopedia performance
- **Storage:** 2GB available storage for application and ML models
- **Hardware:** Camera and microphone required for complete functionality

Recommended Specifications

- **Android:** Version 10.0 or higher
- **iOS:** Version 14.0 or higher
- **RAM:** 4GB or more
- **Storage:** 4GB available storage
- **Processor:** Modern ARM architecture (2020 or later)

.1.2. Machine Learning Model Specifications

Image Classification

- **Object Detection:** SSD MobileNet V1 (MLKit)
- **Bird Classification:** Custom MobileNetV2 Model

- **Input Format:** RGB images, variable dimensions
- **Output:** Classification probabilities with bounding box coordinates

Audio Classification

- **Primary Model:** BirdNET_GLOBAL_6K_V2.4_Model_FP32.tflite
- **Meta Model:** BirdNET_GLOBAL_6K_V2.4_MData_Model_FP16.tflite
- **Input Format:** 48kHz, mono, 3-second audio samples
- **Output:** 6,522 bird species classification probabilities
- **Model Size:** Approximately 25-30 MB combined

.2. Development Environment Setup

.2.1. Prerequisites

```

1 {
2   "expo": "~51.0.39",
3   "react": "18.2.0",
4   "react-native": "0.74.5",
5   "@tensorflow/tfjs": "^4.22.0",
6   "@tensorflow/tfjs-react-native": "^1.0.0",
7   "react-native-vision-camera": "^4.6.4",
8   "@infinitered/react-native-mlkit-object-detection": "^3.1.0",
9   "@infinitered/react-native-mlkit-image-labeling": "^3.1.0",
10  "expo-sqlite": "~15.0.0",
11  "firebase": "^11.6.1",
12  "react-native-fast-tflite": "^1.6.1"
13 }

```

Listing 1: Package Versions

.2.2. Development Commands

```

1  # TypeScript type checking
2  npm run typecheck
3
4  # Start development server
5  npm start
6
7  # Run on Android device
8  npm run android:device
9
10 # Run on iOS simulator
11 npm run ios
12
13 # Execute test suite
14 npm test
15
16 # Production build generation
17 npm run build

```

Listing 2: Development Scripts

.3. Database Architecture and Schema Design

.3.1. SQLite Database Schema Implementation

```

1  -- Bird observation records (main spotting table)
2  CREATE TABLE bird_spottings (
3      id                INTEGER PRIMARY KEY AUTOINCREMENT,
4      image_uri         TEXT,
5      video_uri         TEXT,
6      audio_uri         TEXT,
7      text_note         TEXT,
8      gps_lat           REAL,
9      gps_lng           REAL,
10     date              TEXT,
11     bird_type         TEXT,
12     image_prediction  TEXT,
13     audio_prediction  TEXT,
14     synced            INTEGER DEFAULT 0,
15     latinBirDex       TEXT
16 );
17
18 -- BirDex species database (30,000+ species)
19 CREATE TABLE birddex (
20     species_code       TEXT    PRIMARY KEY,

```

```

21     english_name      TEXT      NOT NULL,
22     scientific_name   TEXT      NOT NULL,
23     category          TEXT,
24     family            TEXT,
25     order_            TEXT,
26     sort_v2024        TEXT,
27     clements_v2024b_change TEXT,
28     text_for_website_v2024b TEXT,
29     range             TEXT,
30     extinct           TEXT,
31     extinct_year      TEXT,
32     sort_v2023        TEXT,
33     de_name           TEXT,
34     es_name           TEXT,
35     ukrainian_name    TEXT,
36     ar_name           TEXT
37 );
38
39 -- Database metadata and versioning
40 CREATE TABLE metadata (
41     key   TEXT PRIMARY KEY,
42     value TEXT
43 );
44
45 -- Indexes for performance optimization
46 CREATE INDEX idx_bird_spotting_date ON bird_spotting(date);
47 CREATE INDEX idx_bird_spotting_synced ON bird_spotting(synced);
48 CREATE INDEX idx_birddex_english ON birddex(english_name COLLATE NOCASE);
49 CREATE INDEX idx_birddex_scientific ON birddex(scientific_name COLLATE NOCASE);
50 CREATE INDEX idx_birddex_category ON birddex(category);
51 CREATE INDEX idx_birddex_family ON birddex(family);

```

Listing 3: Database Schema

.4. Cloud Architecture and API Specifications

.4.1. Firebase Firestore Data Collections

```

1 // User profile management
2 users/{userId} {
3     email: string,
4     displayName: string,
5     createdAt: timestamp,
6     settings: {
7         language: string,

```

```

8         theme: string,
9         notifications: boolean
10    }
11 }
12
13 // Bird observation records
14 observations/{observationId} {
15     userId: string,
16     speciesName: string,
17     commonName: string,
18     scientificName: string,
19     confidence: number,
20     timestamp: timestamp,
21     location: geopoint,
22     imageUrl?: string,
23     audioUrl?: string,
24     notes?: string,
25     metadata: {
26         appVersion: string,
27         deviceInfo: string
28     }
29 }
30
31 // Media file management
32 media/{mediaId} {
33     observationId: string,
34     type: 'image' | 'audio',
35     fileName: string,
36     size: number,
37     uploadedAt: timestamp,
38     downloadUrl: string
39 }

```

Listing 4: Firestore Collections

.5. Performance Evaluation and System Benchmarks

.5.1. Machine Learning Pipeline Performance Analysis

Operation	Low-End Device	Mid-Range Device	High-End Device
Photo capture	800ms	500ms	300ms
Object detection	1200ms	800ms	400ms
Image classification	2000ms	1200ms	600ms
Audio recording	3000ms	3000ms	3000ms
Audio classification	1500ms	1000ms	500ms
Complete cycle	8.5s	6.5s	4.8s

Tab. 1.: Machine Learning Pipeline Performance by Device Classification

.5.2. Memory Utilization Analysis

System Component	Memory Utilization
Application base	50 MB
ML model framework	30 MB
BirDex database	25 MB
Image cache system	100 MB (configurable)
User-generated data	Variable
Minimum total	105 MB

Tab. 2.: Memory Utilization by System Components

.6. System Configuration Framework

.6.1. Application Configuration Parameters

```
1 export const Config = {
2   // Machine learning pipeline configuration
3   camera: {
4     pipelineDelay: 2, // Seconds between processing cycles
5     confidenceThreshold: 0.7, // Minimum confidence for data persistence
6     maxDetections: 5, // Maximum concurrent detection objects
7     imageQuality: 0.3, // Image quality for ML processing optimization
8   },
9 }
```



```

10 // Audio processing configuration
11 audio: {
12     recordingDuration: 3000, // Recording duration in milliseconds
13     sampleRate: 48000, // Sample rate in Hz
14     channels: 1, // Mono channel configuration
15     format: 'wav',
16 },
17
18 // Database management configuration
19 database: {
20     pageSize: 20, // Elements per pagination page
21     cacheSize: 100, // Cache size in MB for image storage
22     syncInterval: 300000, // Synchronization interval: 5 minutes in ms
23 },
24
25 // User interface configuration
26 ui: {
27     theme: 'auto', // Theme options: 'light', 'dark', 'auto'
28     language: 'auto', // Language code or 'auto' detection
29     hapticFeedback: true,
30     animations: true,
31 }
32 };

```

Listing 5: Configuration Options

.7. System Diagnostics and Troubleshooting Framework

.7.1. Common System Issues and Resolution Procedures

Machine Learning Pipeline Initialization Failure

Symptoms: No ML processing activity, UI displays Offline status

Resolution procedures:

1. Verify camera permission grants
2. Validate ML model loading integrity
3. Execute application restart procedure

4. Confirm device storage availability (minimum 1GB free space)

Audio Classification System Malfunction

Symptoms: Absence of audio predictions in HUD display

Resolution procedures:

1. Verify microphone permission configuration
2. Test hardware microphone accessibility
3. Validate BirdNET model loading status
4. Confirm audio format compatibility requirements

System Performance Degradation

Symptoms: Reduced processing speed, elevated memory utilization

Optimization procedures:

1. Increase `Config.camera.pipelineDelay` parameter
2. Reduce image quality settings for ML processing
3. Decrease cache allocation size
4. Terminate background application processes

.7.2. Diagnostic Command Interface

```
1 // Pipeline logging monitoring
2 # Search for diagnostic patterns:
3 [UnifiedPipeline] State: capturing_image
4 [UnifiedPipeline] State: detecting_objects
5 [UnifiedPipeline] State: recording_audio
6
```

```

7 // Error validation procedures
8 [UnifiedPipeline] image error: [Error details]
9 [UnifiedPipeline] audio error: [Error details]
10
11 // Performance metric analysis
12 [Performance] Photo capture time: 500ms
13 [Performance] Detection count: 3
14 [Performance] Audio processing: 1000ms

```

Listing 6: Debug Commands

.8. Production Deployment Framework

.8.1. Android Application Package Build Process

```

1 # Dependency installation procedure
2 npm install --legacy-peer-deps
3
4 # Native project generation
5 npx expo prebuild --clean --platform android
6
7 # Local build process (when EAS unavailable)
8 npx expo run:android --device
9
10 # EAS build process (recommended approach)
11 eas build --platform android --profile production

```

Listing 7: Android Build Process

.8.2. iOS Application Build Framework

```

1 # Dependency installation procedure
2 npm install --legacy-peer-deps
3
4 # Native project initialization
5 npx expo prebuild --clean --platform ios
6
7 # EAS build process
8 eas build --platform ios --profile production

```

Listing 8: iOS Build Process

.9. Licensing Framework and Attribution

.9.1. Open Source License Compliance

- **LogChirpy:** AGPL-3.0 License
- **React Native:** MIT License
- **Expo:** MIT License
- **TensorFlow.js:** Apache 2.0 License
- **BirdNET Models:** Custom Academic License
- **MLKit:** Google Terms of Service

.9.2. Data Source Attribution

- **BirDex Database:** Compiled from comprehensive ornithological sources
- **Wikipedia:** Species information and taxonomic imagery
- **Macaulay Library:** Reference media for avian species validation
- **OpenStreetMap:** Cartographic data and geospatial services

.9.3. External Technical Contributions

- **BirdNET Team:** Original audio classification models (.h5 source files)
- **whoBIRD Project:** Original Kotlin implementation (ported to React Native by team)
- **Infinite Red:** MLKit React Native packages
- **Marc Wannenmacher:** React Native Vision Camera library development

A. Team Collaboration Framework and Development Contributions

The LogChirpy project was executed by a team comprising three students over a 12-week development cycle (Q2-Q3 2025).

A.1. Individual Contributions and Research Responsibilities

A.1.1. Martin Lauterbach

Primary Research Responsibilities

- System architecture design and technical leadership
- Machine learning pipeline development and integration
- Custom user interface components and design system implementation
- Database schema design and internationalization framework
- Performance optimization and systematic testing methodologies

Development Role and Scope Martin served as the primary architect and developer, handling the majority of system implementation:

Core Responsibilities:

- **Technical Leadership:** System architecture design and development methodology

- **ML Model Engineering:** Converting .h5 models to .tflite, porting WhoBird from Kotlin to React Native
- **Full-Stack Development:** Frontend UI, backend services, database design, and integration
- **DevOps & Testing:** CI/CD pipeline, build automation, comprehensive test suite (>90% coverage)
- **Data Engineering:** 30,000+ species database, mass translation system, Wikimedia image scraping and local bundling
- **Performance Optimization:** Mobile-specific optimizations, sequential ML pipeline architecture

Areas of Specialization

- **Machine Learning Model Conversion:** .h5 to .tflite conversion, model labeling, Who-Bird Kotlin-to-React Native porting
- **ML Pipeline Architecture:** Sequential processing orchestration, unified image/audio processing, TensorFlow.js integration
- **Mobile Development Architecture:** React Native optimization, Expo framework utilization, performance enhancement
- **Database System Design:** SQLite architecture, species mapping algorithms, internationalization implementation
- **DevOps and Automation:** CI/CD pipeline development, build automation, Windows-WSL2 integration

Development Effort Allocation Estimated development contribution: 85% of total project implementation time

- Architecture and system design: 40 hours
- ML integration and pipeline development: 60 hours
- User interface and experience development: 50 hours
- Database design and internationalization: 45 hours

- Testing methodologies and optimization: 35 hours
- **Total Contribution: 230 hours**

A.1.2. Luis Wehrberger

Primary Research Responsibilities

- Firebase integration and cloud services architecture
- User authentication and management system implementation
- Cloud synchronization and file upload infrastructure
- First Google-Maps Geographic information system integration, later changed to Open-StreetMap by Martin
- Quality assurance and code optimization initiatives

Development Role and Scope Luis focused on cloud infrastructure and system integration as well as bug fixes and refactoring:

Core Responsibilities:

- **Cloud Services:** Firebase Authentication, Firestore configuration, cloud synchronization
- **System Stability:** Bug fixes, dependency management, architecture issue resolution
- **UI Components:** Settings redesign, map integration, modal log systems
- **Quality Assurance:** Package optimization, migration to modern camera packages, bug fixes and refactoring

Areas of Specialization

- **Cloud Services Architecture:** Firebase Authentication, Firestore implementation, Storage systems

- **Code Quality Assurance:** Refactoring methodologies, dependency management optimization
- **User Interface Enhancement:** Settings design optimization, map visualization implementation

Development Effort Allocation Estimated development contribution: 12% of total project implementation time

- Firebase setup and integration: 14 hours
- Authentication system development: 10 hours
- Map integration and geospatial features: 8 hours
- Bug resolution and refactoring initiatives: 16 hours
- Camera and media handling improvements: 10 hours
- **Total Contribution: 58 hours**

A.1.3. Youmna Samounah

Primary Research Responsibilities

- Initial wireframe development and conceptual design
- Support in localization
- Design feedback provision and usability assessment

Development Role and Scope Youmna contributed to design and specific feature development:

Core Responsibilities:

- **Design Input:** Initial wireframes and user experience feedback
- **Feature Development:** Dynamic quiz implementation using BirDex database

- **Localization Support:** Translation assistance and internationalization input

Areas of Specialization

- **User Experience Design:** Wireframing methodologies, user interface conceptualization
- **Internationalization:** Translation protocols, multilingual support systems

Development Effort Allocation Estimated development contribution: 3% of total project implementation time

- Wireframe development: 4 hours
- Internationalization contributions: 2 hours
- Design feedback and evaluation: 3 hours
- Development of the tutorial with animations (fully translated) and video (with subtitles): 11 hours
- Development of the quiz feature (fully translated): 10 hours
- **Total Contribution: 30 hours**

A.2. Collaborative Framework and Team Coordination

Work Distribution Strategy The development workload distribution reflected varying levels of engagement and technical expertise, with Martin handling the majority of core development tasks:

Development Domain	Martin	Luis	Younna
System Architecture	Primary	-	-
ML Model Conversion & Integration	Primary	-	-
Frontend Development	Primary	Minor Support	Design Input
Firebase/Cloud Systems	Minor Support	Primary	-
Database Design & Implementation	Primary	Minor Support	-
User Interface & Theming	Primary	Minor Support	Design Input
Testing & Optimization	Primary	Minor Support	-
Technical Documentation	Primary	Minor Support	Localization Support
Feature Development	Primary	Minor Support	Quiz Feature

Tab. A.1.: Development Domain Responsibility Matrix

Communication and Coordination Protocols

- **Regular Development Meetings:** Weekly status updates and planning sessions
- **Version Control Workflow:** Feature-branch methodology with comprehensive code reviews
- **Documentation Standards:** Comprehensive README files and inline code documentation
- **Issue Management:** GitHub Issues system for bug reporting and feature request tracking

Collaborative Challenges and Solutions

- **Significantly Unequal Contribution Levels:** Martin carried 85% of development workload while other team members had limited availability
- **Technical Complexity Concentration:** Advanced ML model conversion and integration required specialized knowledge concentrated in one team member
- **Project Scale vs Team Capacity:** Ambitious project scope exceeded available team resources and time commitment levels

Successful Collaboration Aspects

- **Focused Expertise Areas:** Luis successfully handled Firebase/cloud integration, Youmna contributed quiz feature

- **Clear Role Definition:** Early recognition of capacity limitations led to realistic task allocation
- **Quality Over Quantity:** Concentrated effort on core features rather than spreading resources thin

A.3. Project Management Methodology

Development Approach

- **Agile Development Methodology:** Iterative feature development with regular release cycles
- **Feature-Driven Development:** Implementation based on user stories and acceptance criteria
- **Continuous Integration:** Automated build processes and testing protocols

Project Timeline and Milestone Framework

Development Milestone	Timeline	Primary Responsibility
Project Initialization	Week 1-2	Martin, Luis
Firebase Integration	Week 1-2	
Martin		
ML Pipeline Development	Week 1-8	Martin
Database Design	Week 2-4	Martin, Luis
UI/UX Implementation	Week 6-10	Martin
Testing and Optimization	Week 9-11	Martin, Luis
Quiz Feature Development	Week 11	Youmna
Documentation Completion	Week 12	Martin

Tab. A.2.: Project Timeline and Responsibility Matrix

A.4. Recommendations for Future Research Projects

Team Composition Optimization

- Pursue more balanced workload distribution among team members

- Establish clear role definitions and responsibility boundaries
- Implement regular communication protocols and progress check-ins

Project Management Enhancement

- Develop detailed project planning with realistic time estimation methodologies
- Implement early risk identification and dependency management protocols
- Establish continuous quality assurance frameworks from project initiation

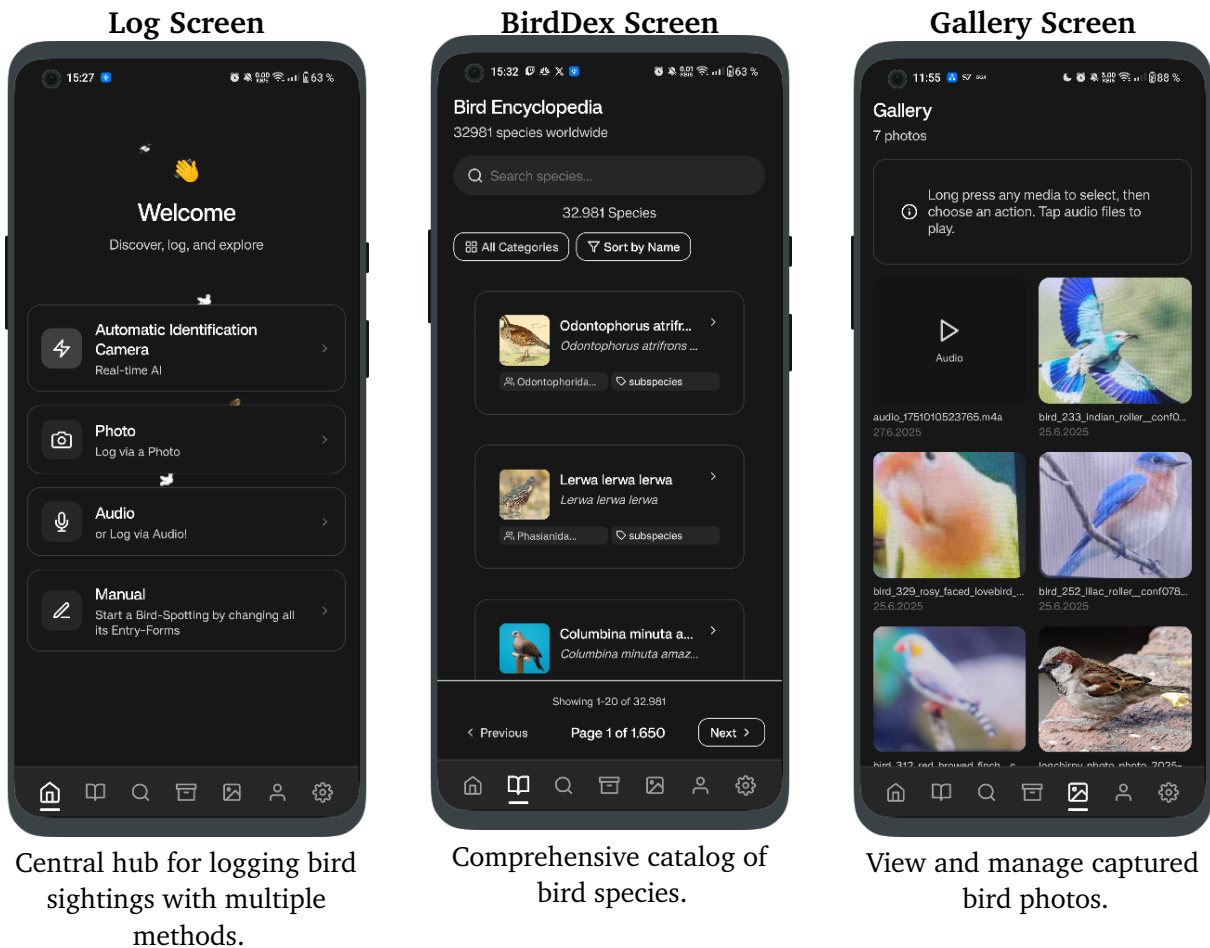
Technical Development Improvements

- Implement early prototyping strategies for critical feature validation
- Establish comprehensive testing strategies from development beginning
- Implement regular code review protocols and pair programming methodologies

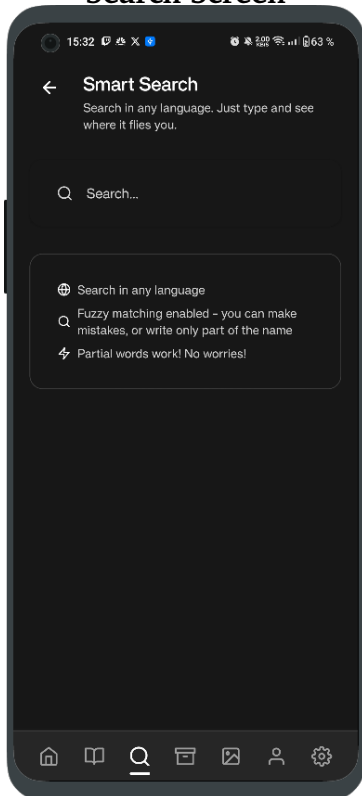
B. Application Screens

This chapter presents the main screens of the LogChirpy application, showcasing its user interface and core functionalities.

B.1. Main Screens

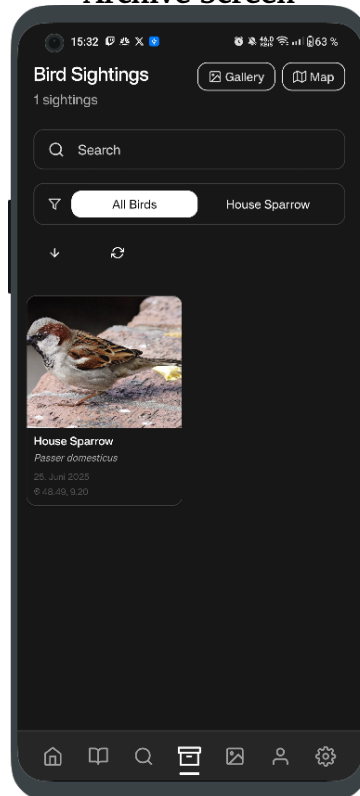


Search Screen



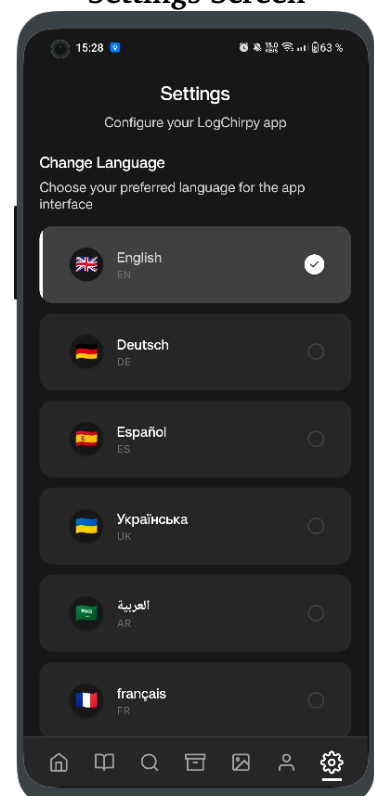
Find specific bird species quickly.

Archive Screen



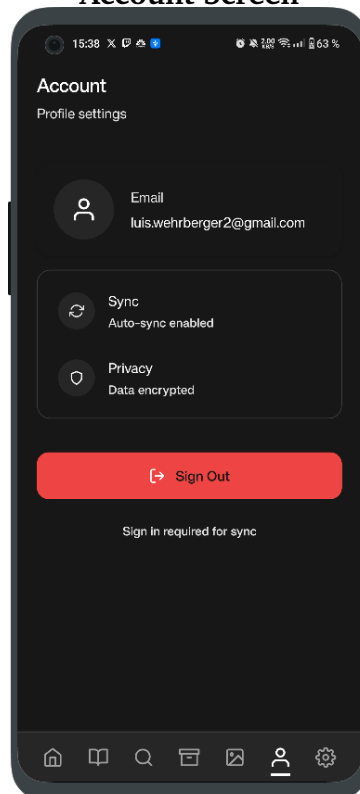
Historical record of bird observations.

Settings Screen



Configure app preferences and options.

Account Screen

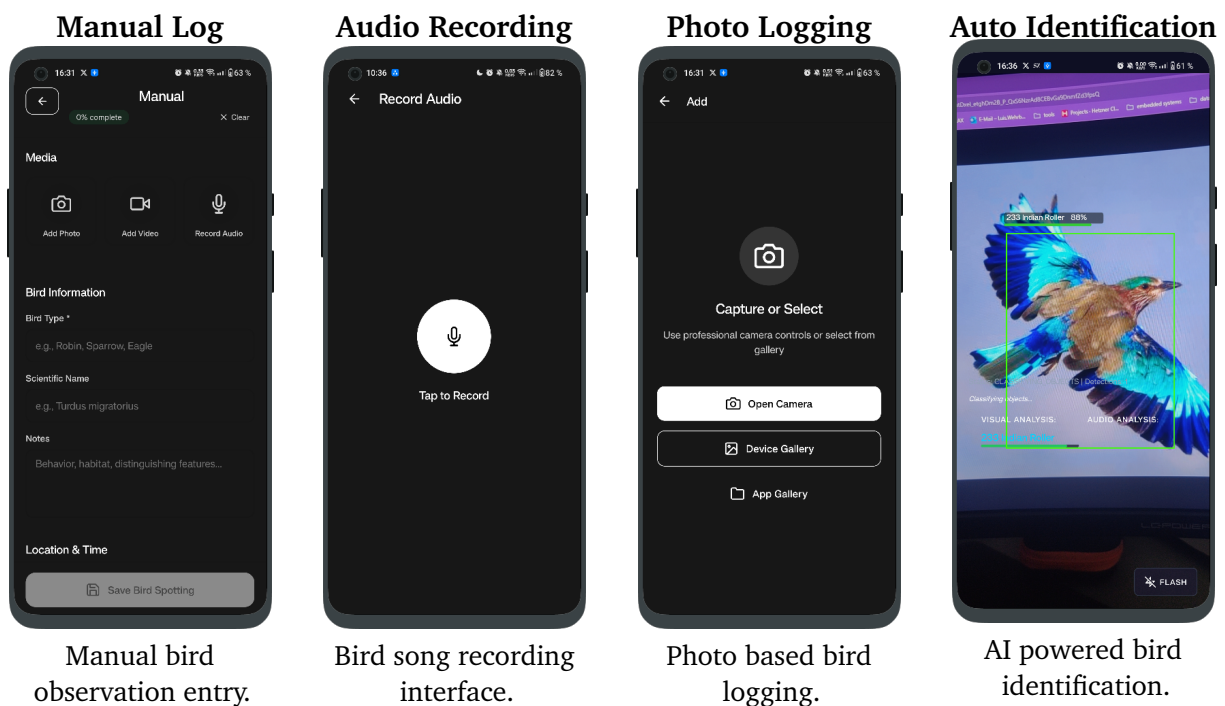


User profile and account management.

B.2. Feature Screens

B.2.1. Logging Features

The logging section offers various methods to record bird observations. Users can choose between manual entry, audio recording, photo based identification, and AI assisted automatic identification. This multimodal approach ensures accurate bird species identification regardless of the observation conditions or user expertise level.



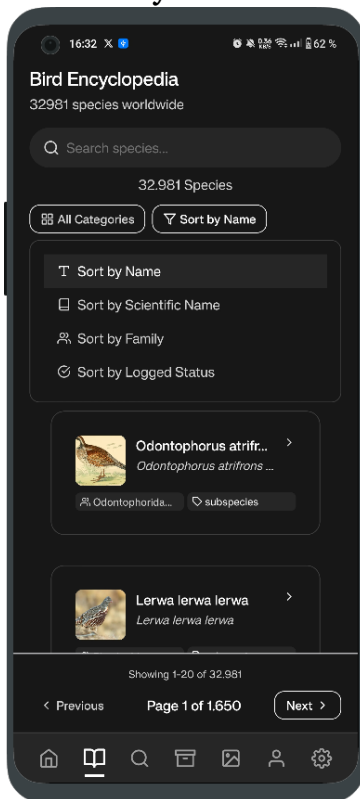
Key aspects of the logging system:

- **Multiple Input Methods:** Choose between manual, audio, and photo logging
- **AI Assistance:** Advanced machine learning for species identification
- **Location Integration:** Automatic GPS tagging of observations
- **Offline Capability:** Full functionality without internet connection
- **Data Validation:** Accuracy checks for species identification

B.2.2. BirdDex Features

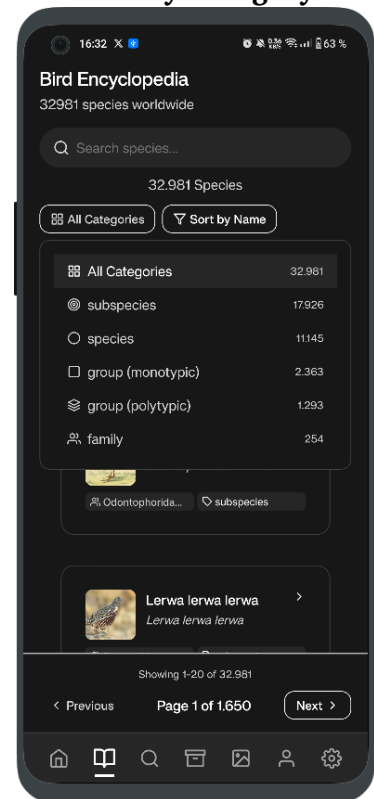
The BirdDex section provides comprehensive bird species information and sorting capabilities. It serves as an educational resource and reference guide, allowing users to explore and learn about different bird species through various organization methods and detailed species information.

Sort by Attribute

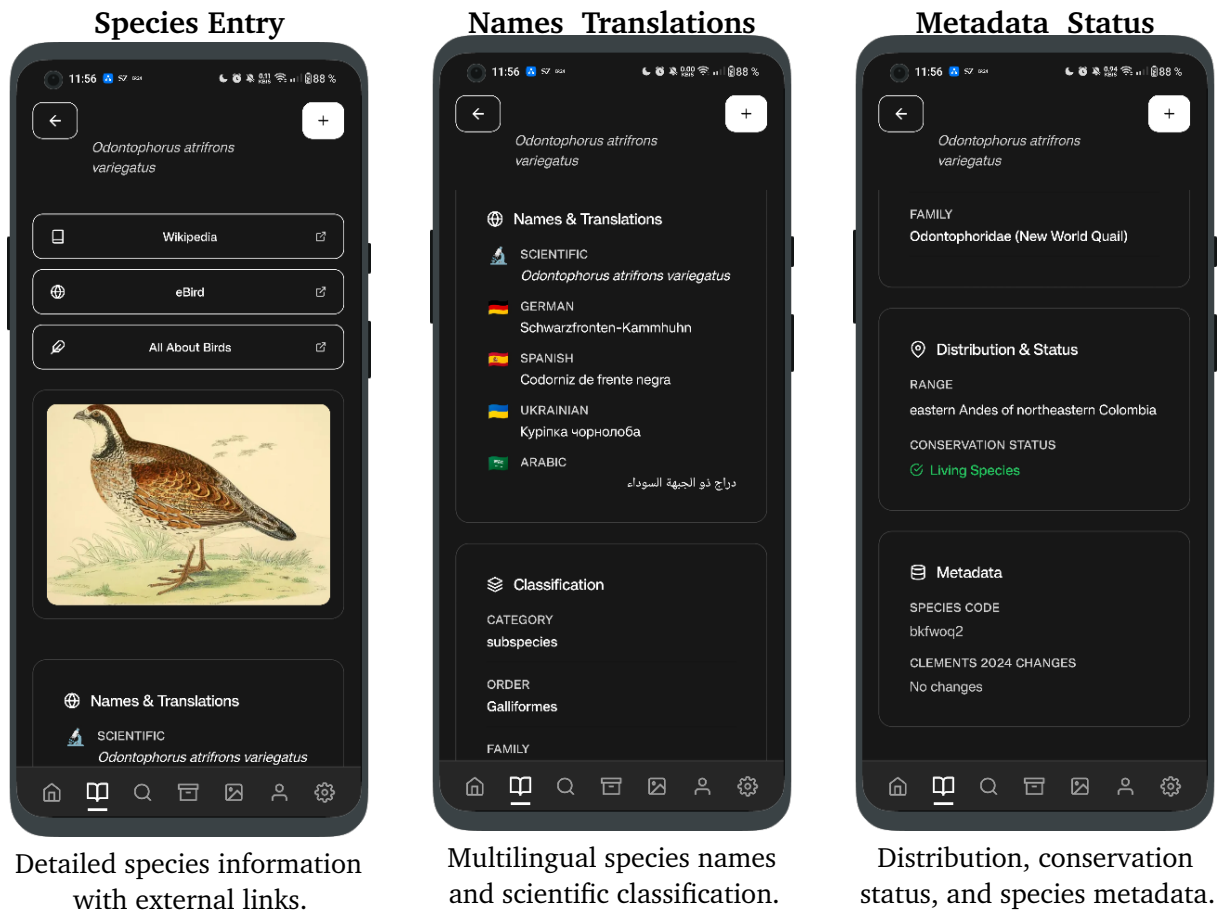


Attribute based species sorting.

Sort by Category



Category based species organization.

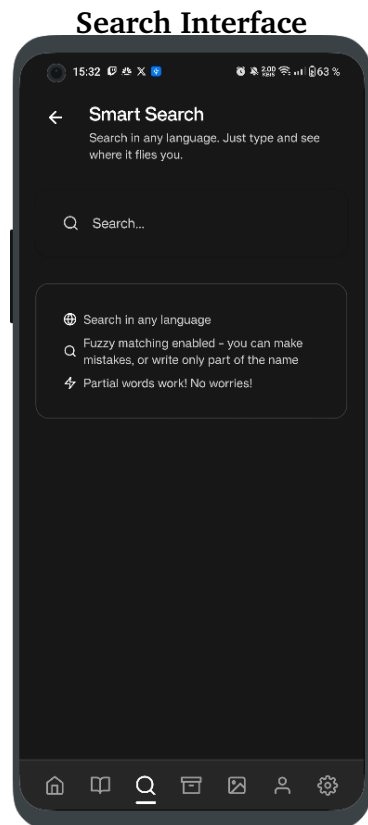


Key aspects of the BirdDex system:

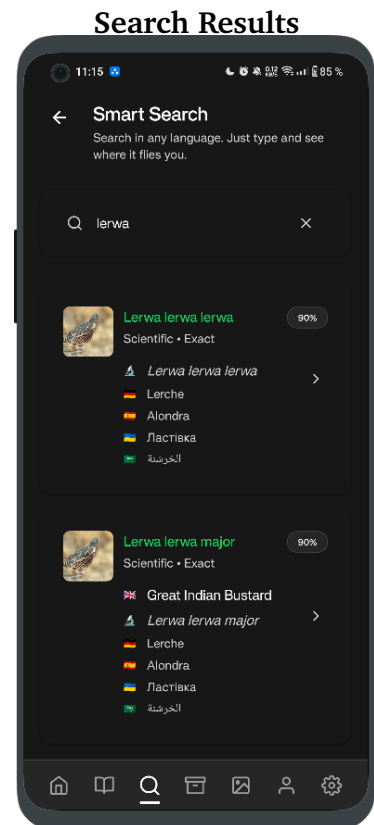
- **Comprehensive Database:** Extensive collection of bird species
- **Multiple Views:** Various ways to organize and explore species
- **Educational Content:** Detailed information about each species
- **Offline Access:** Complete functionality without internet

B.2.3. Search Features

The search functionality allows users to quickly find specific bird species using various search criteria. This feature enhances the user experience by providing fast access to bird information and simplifying the identification process.



Find specific bird species quickly.



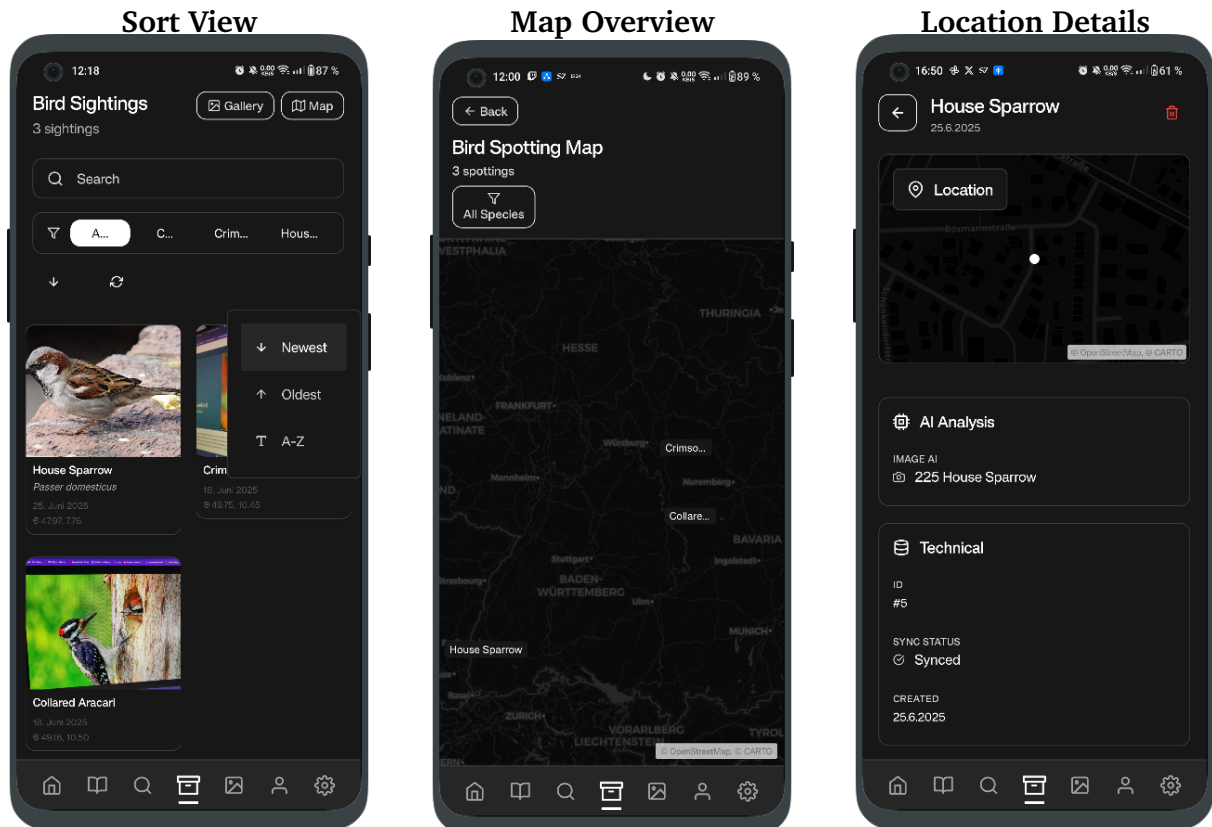
Multilingual search results with species details.

Key aspects of the search system:

- **Fast Search:** Quick species lookup functionality
- **Multiple Criteria:** Search by name, category, or attributes
- **Fuzzy Matching:** You can make mistakes or write only part of the name

B.2.4. Archive Features

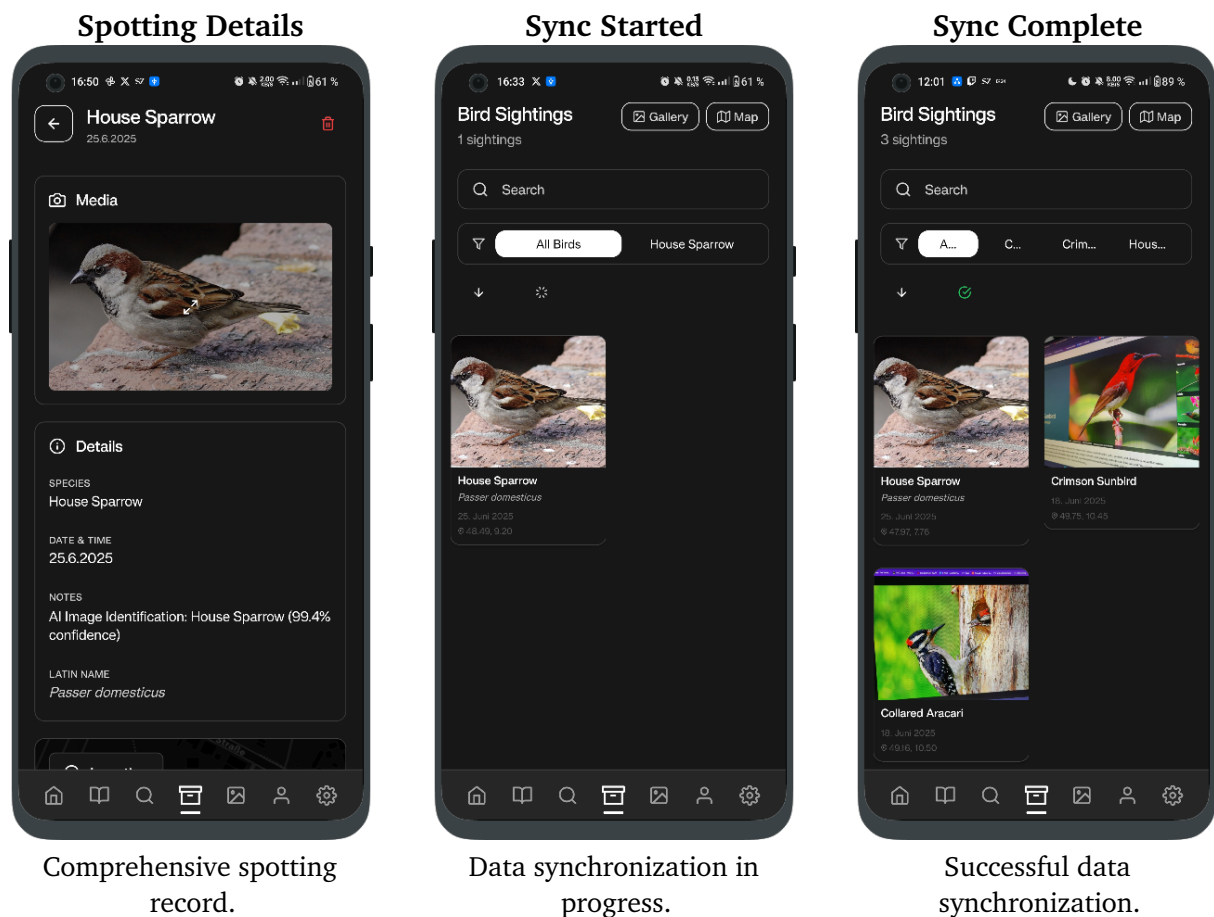
The archive section provides comprehensive management of past observations. It offers multiple views and organization options for reviewing historical bird sightings, including list views, map visualization, and detailed spotting records. All data is stored locally with optional cloud backup when signed in.



Sorting and filtering options.

Geographical view of spotting locations.

Detailed location and AI analysis.



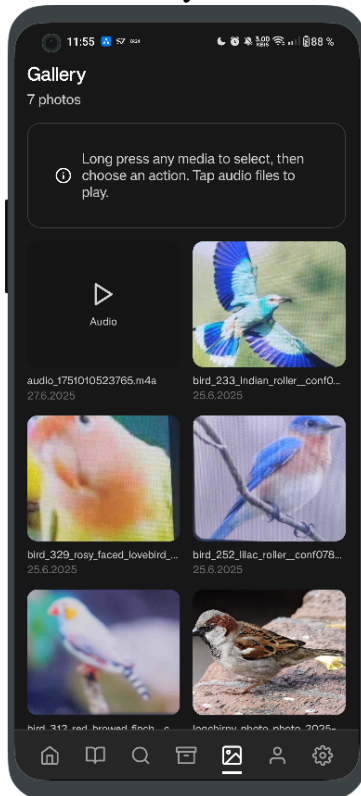
Key aspects of the archive system:

- **Flexible Organization:** Multiple sorting and filtering options
- **Spatial Visualization:** Map based view of observation locations
- **Data Management:** Local storage with cloud backup option
- **Detailed Records:** Comprehensive information for each sighting
- **Export Capabilities:** Share and backup observation data

B.2.5. Gallery Features

The gallery section provides a visual interface for managing and viewing captured bird photos and audio recordings. Users can browse their collection, organize, and integrate photos or audio recordings with their bird observations for comprehensive documentation.

Gallery View



View and manage captured bird photos or audio recordings.

Gallery Options

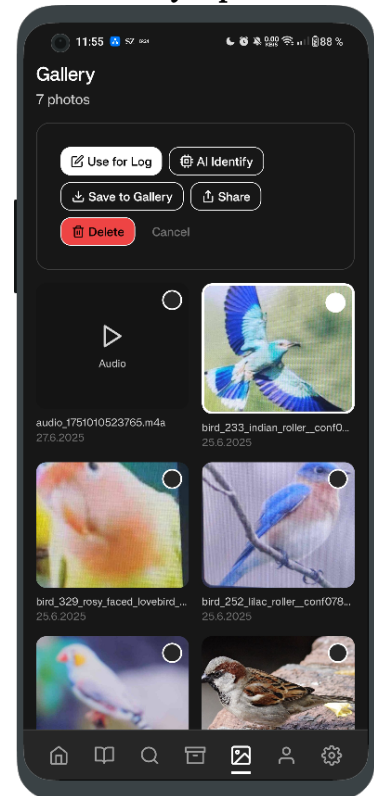


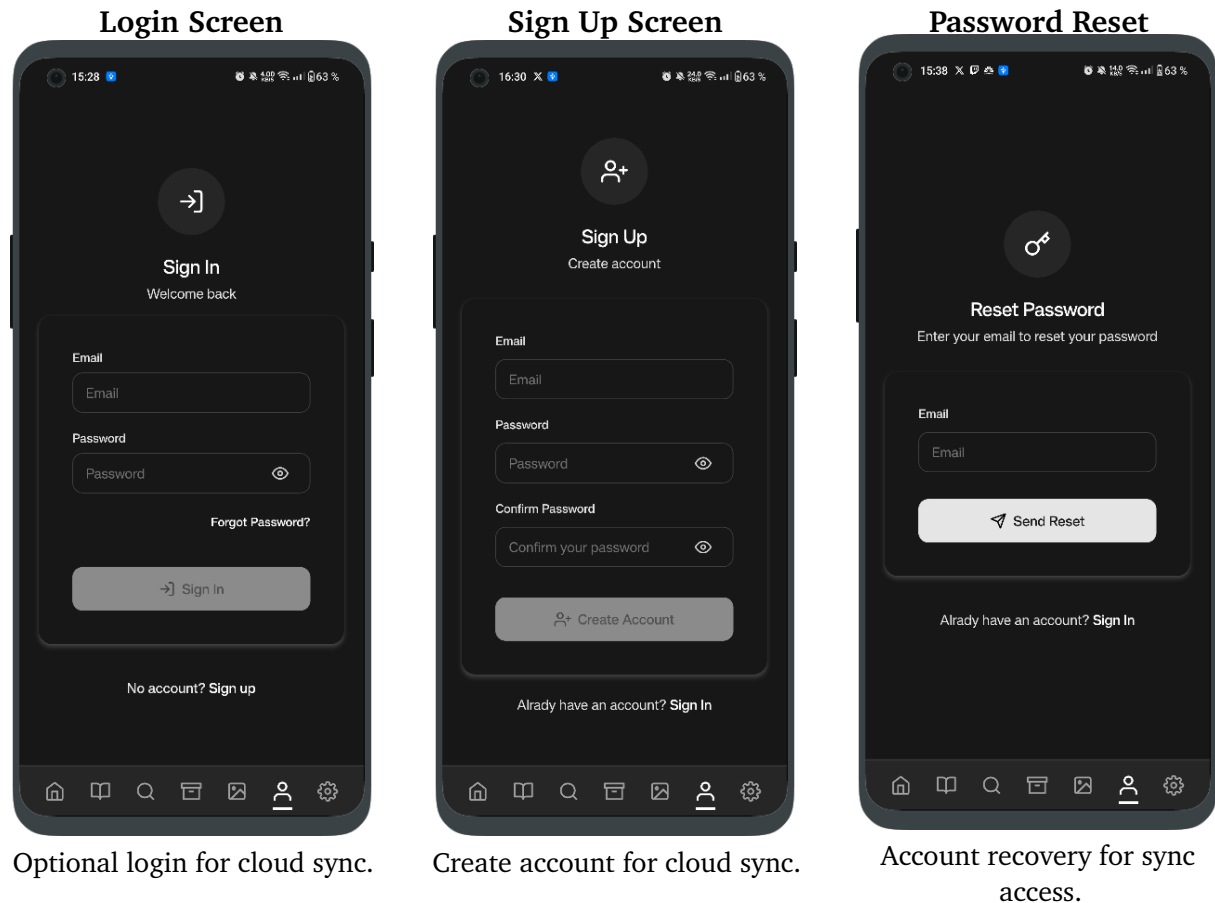
Photo management actions and AI tools.

Key aspects of the gallery system:

- **Photo Management:** Organize and browse bird photos and audio recordings
- **Use for Log:** Initiate logging process
- **AI Identification:** Identify bird species right from the gallery
- **Local Storage:** All photos stored locally on device
- **Sharing Options:** Export and share photos and audio recordings

B.2.6. Account Features

The account section provides user authentication for syncing bird logs across devices. All other app features work completely offline without requiring login. This design ensures maximum accessibility while offering optional cloud synchronization for users who want to access their bird spotting data on multiple devices.

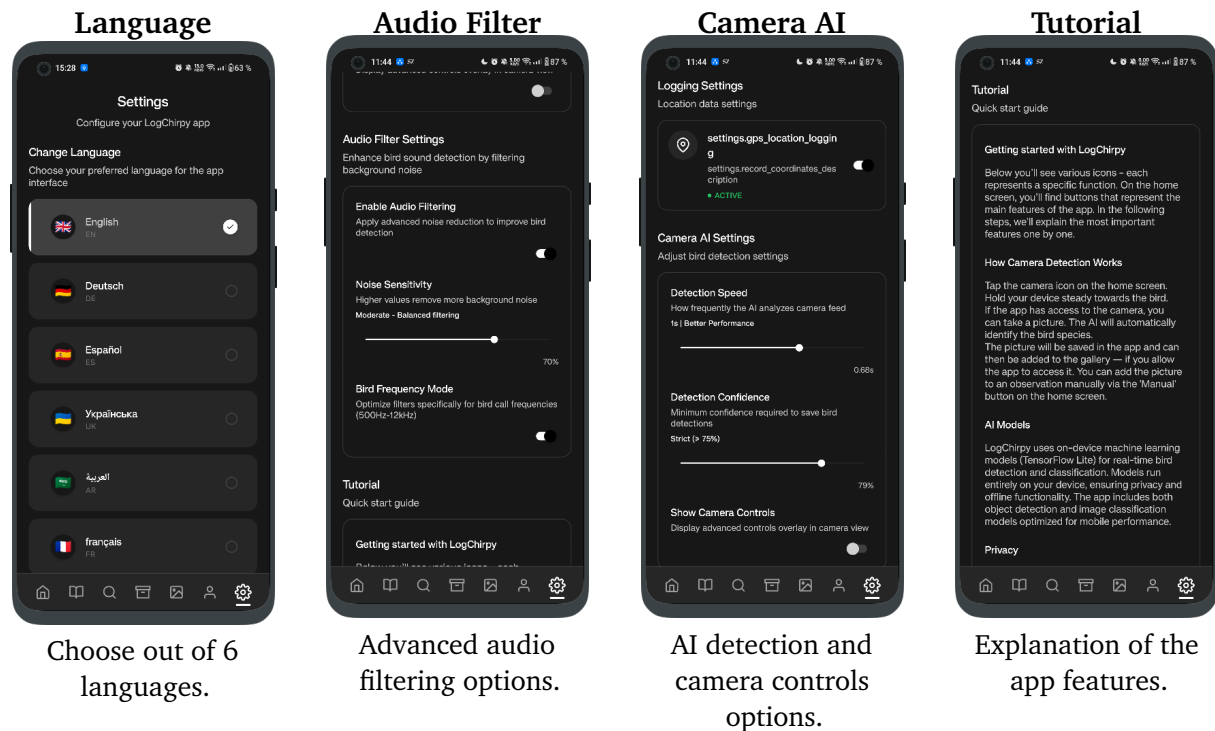


Key aspects of the account system:

- **Optional Authentication:** Login is only required for cloud sync
- **Offline First:** All features work without authentication, except sync
- **Sync Management:** Control when and how data syncs
- **Data Privacy:** Encrypted cloud storage when syncing
- **Cross Device Access:** Access logs on multiple devices when logged in

B.2.7. Settings Features

The settings section allows users to configure app preferences, customize the user experience, and manage application behavior. This includes display options, notification settings, data management preferences, and accessibility features.



Key aspects of the settings system:

- **Language Support:** Choose from 6 languages with flag indicators
- **Audio Filtering:** Advanced noise reduction and bird frequency optimization
- **Camera AI Configuration:** Adjustable detection speed, confidence thresholds, and location tracking
- **Tutorial System:** Comprehensive getting started guide explaining app features
- **Real time Adjustments:** Live sliders for fine tuning AI performance settings